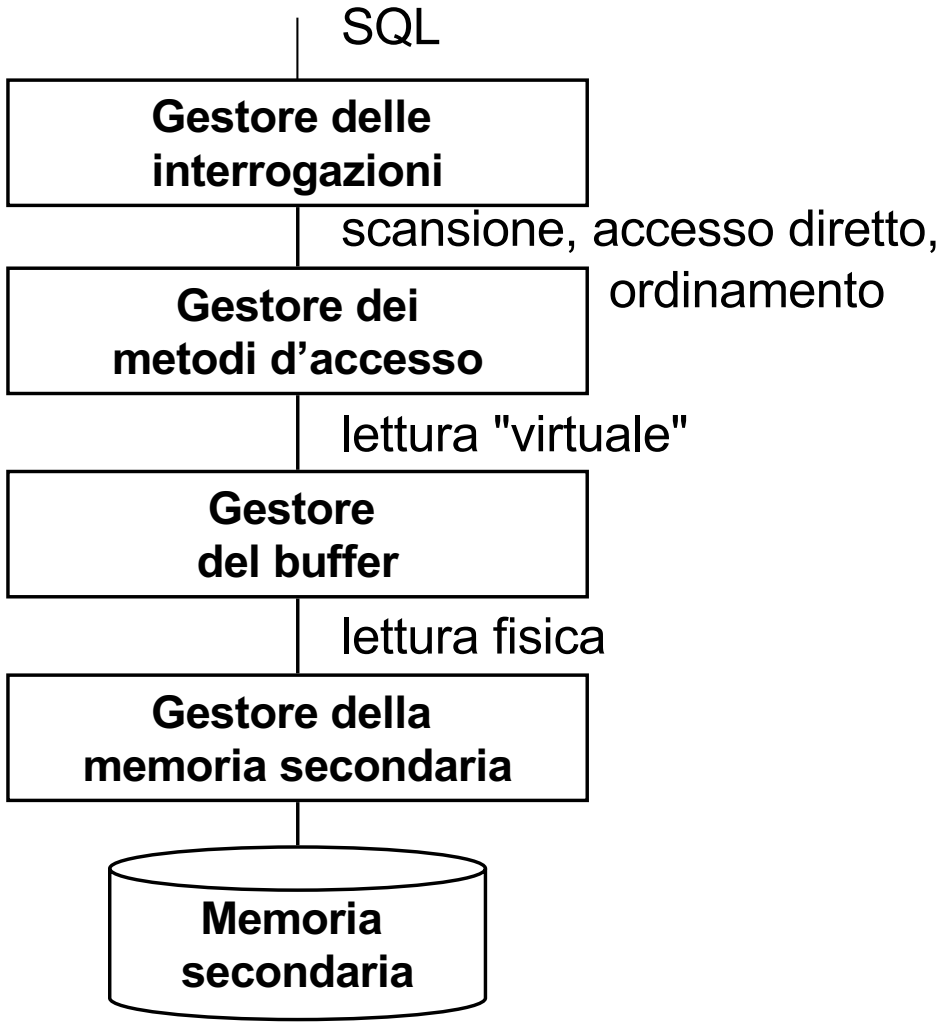
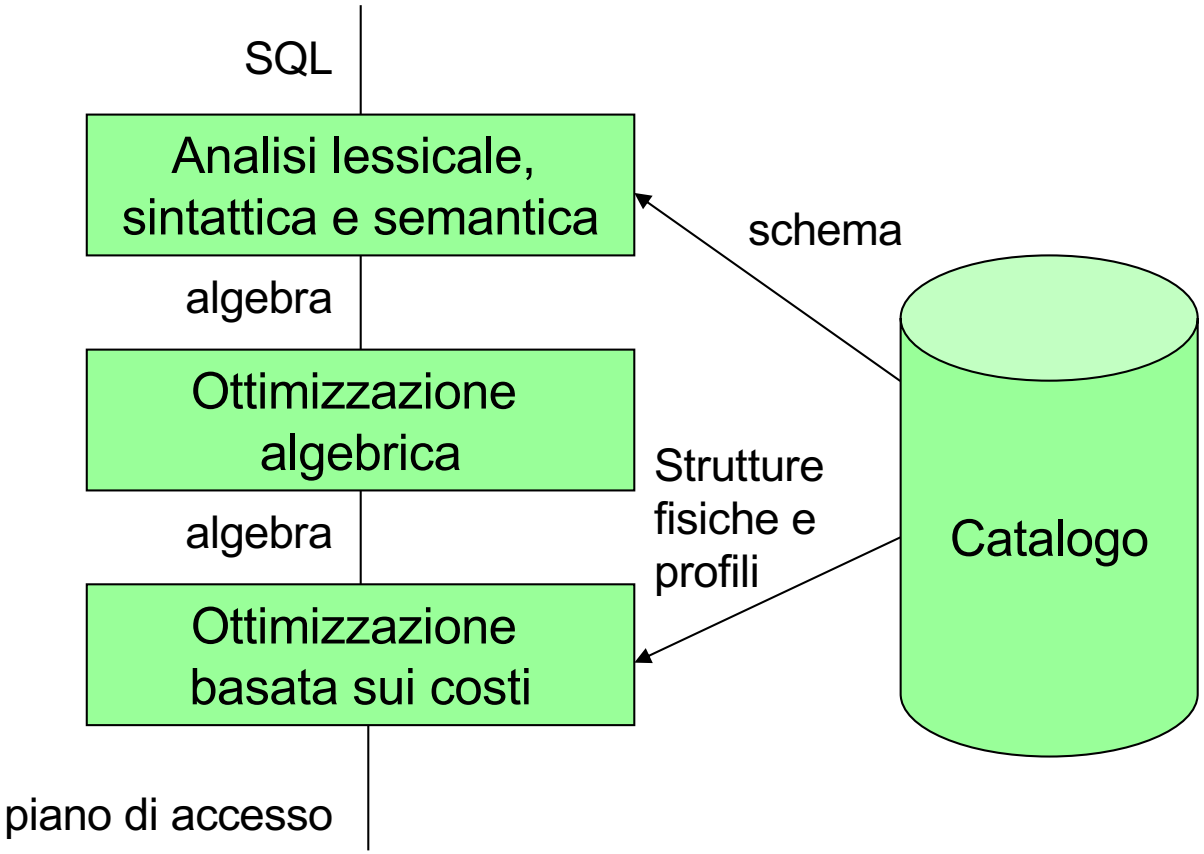
Gestore degli accessi e delle interrogazioni



Esecuzione e ottimizzazione delle interrogazioni

- ***Query processor*** (o ***Ottimizzatore***): un modulo del DBMS
- Motivazione
  - le interrogazioni sono espresse ad alto livello (ricordare il concetto di **indipendenza dei dati**):
    - insiemi di ennuple
    - poca proceduralità
  - l'ottimizzatore sceglie la strategia realizzativa (di solito fra diverse alternative), a partire dall'istruzione (SQL)

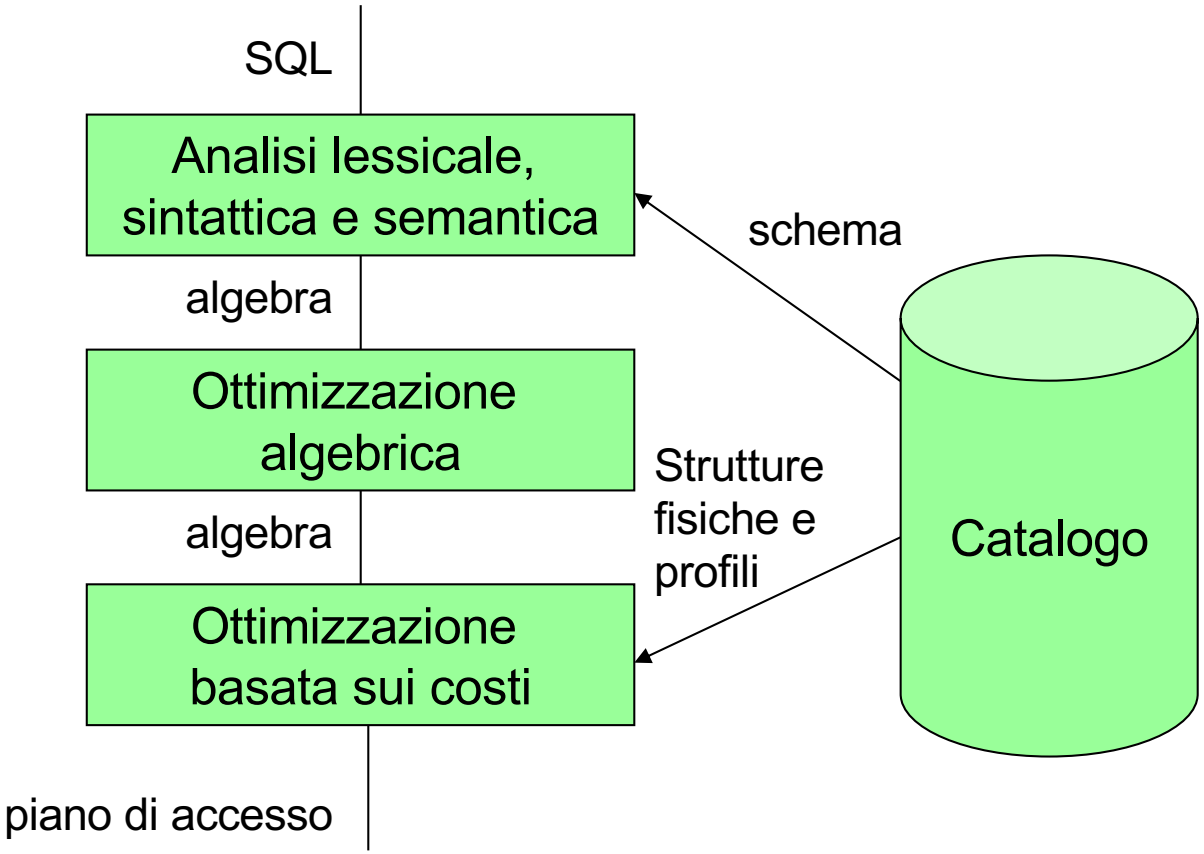
Il processo di esecuzione delle interrogazioni



"Profili" delle relazioni

- Informazioni quantitative:
  - cardinalità di ciascuna relazione
  - dimensioni delle ennuple
  - dimensioni dei valori
  - numero di valori distinti degli attributi
  - valore minimo e massimo di ciascun attributo
  - …
- Sono memorizzate nel "catalogo" e aggiornate con comandi del tipo `update statistics` (o simile)
- Utilizzate nella fase finale dell'ottimizzazione, per stimare I costi delle singole operazioni e le dimensioni dei risultati intermedi

Il processo di esecuzione delle interrogazioni



Esempio su Postgres

R1(ABC)
R2(DEF)
R3(GHI)

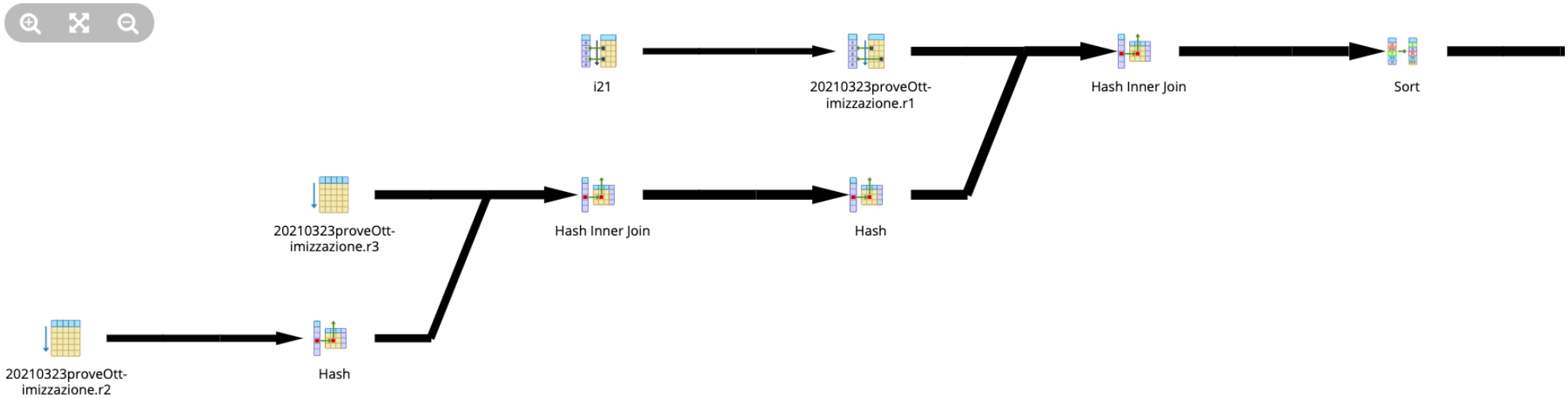
```
set search_path to "20210323proveOttimizzazione";
drop table if exists r1;
create table r1 (       a numeric not null primary key,
                        b numeric,
                        c numeric);
drop table if exists r2;
create table r2 (       d numeric not null primary key,
                        e numeric,
                        f numeric);
drop table if exists r3;
create table r3 (       g numeric not null primary key,
                        h numeric,
                        i numeric);
alter table r1 add constraint fk1 foreign key (c) references r2 (d);
alter table r2 add constraint fk2 foreign key (f) references r3 (g);
create index i12 on r1(B);
```

Esempio su Postgres (2)

- Caricamento dati (vedremo fra qualche lezione)
  - 1.000.000 ennuple R1
  - 100.000 ennuple R2
  - 1.000.000 ennuple R3
- Interrogazione

```
SELECT A , E
FROM    R1, R3, R2
WHERE C=D AND B=20 AND F=G AND I>2
```

Esecuzione in Postgres



Da SQL all'algebra

- (Semplificando)
  - prodotto cartesiano (FROM)
  - selezione (WHERE)
  - proiezione (SELECT)

R1(ABC)

R2(DEF)

R3(GHI)

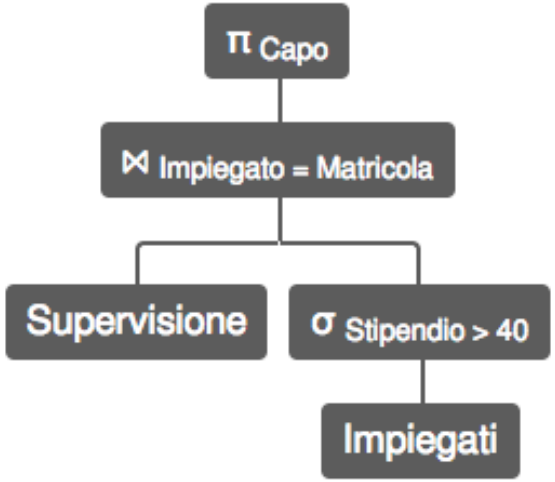
```
SELECT   A , E
FROM     R1, R2, R3
WHERE    C=D AND B=20 AND F=G AND H=7 AND I>2
```

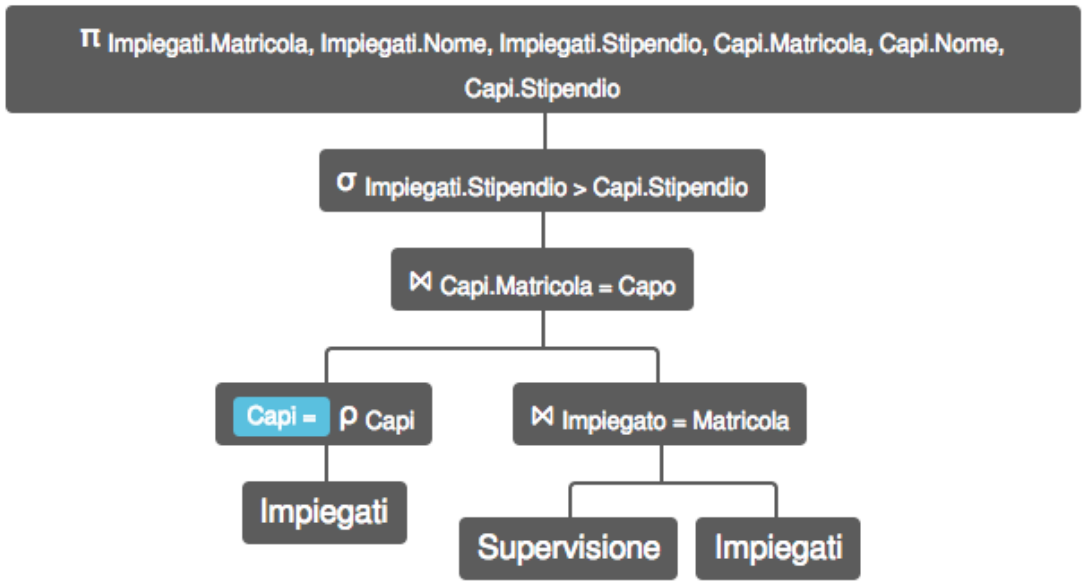
$$\text{PROJ}_{AE}\ (\text{SEL}_{C=D\ \text{AND}\ B>100\ \text{AND}\ F=G\ \text{AND}\ H=7\ \text{AND}\ I>2}\ (\ (R1\ \text{JOIN}\ R2)\ \text{JOIN}\ R3))$$

Rappresentazione ad albero

- Alberi:
  - foglie: dati (relazioni, file)
  - nodi intermedi: operatori (operatori algebrici, poi effettivi operatori di accesso)

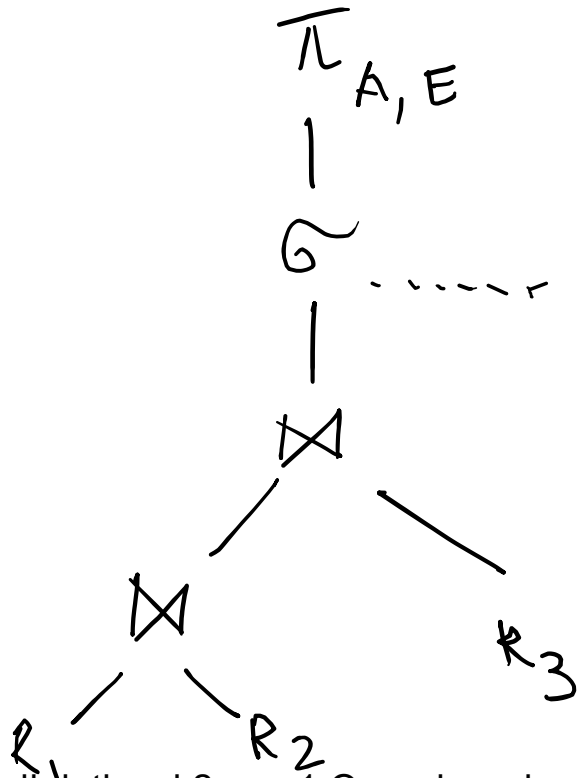
Nel corso di Basi di dati I, RelaX





Da SQL all'algebra

```
SELECT    A , E
FROM      R1, R2, R3
WHERE     C=D AND B>100 AND F=G AND H=7 AND I>2
```



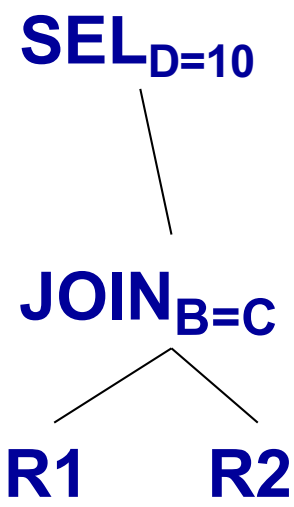
Due interrogazioni ancora più semplici

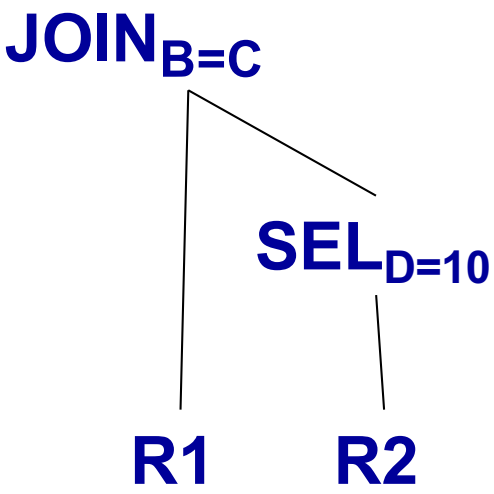
R_1 (A B)

R_2 (C D)

$SEL_{D=10}$ (R_1 $JOIN_{B=C}$ R_2)

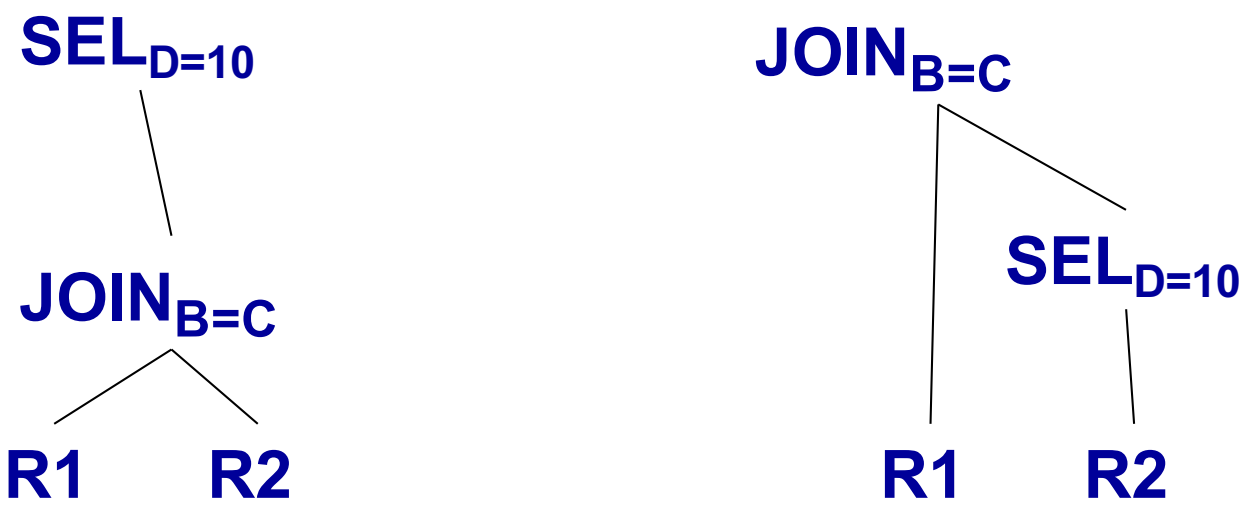
R_1 $JOIN_{B=C}$ $SEL_{D=10}$ (R_2)





Alberi per la rappresentazione di interrogazioni

- Che cosa possiamo dire di queste due espressioni dell'algebra?
- In che cosa sono uguali e in che cosa diverse?



Ottimizzazione algebrica

- Il termine ottimizzazione è improprio (anche se efficace) perché il processo utilizza euristiche
- Si basa sulla nozione di equivalenza:
  - Due espressioni sono equivalenti se producono lo stesso risultato qualunque sia l'istanza attuale della base di dati
- I DBMS cercano di eseguire espressioni equivalenti a quelle date, ma meno "costose"
- Euristica fondamentale:
  - selezioni e proiezioni il più presto possibile (per ridurre le dimensioni dei risultati intermedi):
    - "push selections down"
    - "push projections down"

"Push selections"

R_1 (A B)

R_2 (C D)

$$SEL_{D=10} (R_1 \ JOIN_{B=C} \ R_2) = R_1 \ JOIN_{B=C} \ SEL_{D=10} (R_2)$$

- Riduce in modo significativo la dimensione del risultato intermedio (e quindi il costo dell'operazione)

Euristica per l'ottimizzazione algebrica

- Decomporre le selezioni congiuntive in successive selezioni atomiche
- Anticipare il più possibile le selezioni
- In una sequenza di selezioni, anticipare le più selettive
- Combinare prodotti cartesiani e selezioni per formare join (eventualmente riordinando gli operandi)
- Anticipare il più possibile le proiezioni (anche introducendone di nuove);
  - peraltro questo serve soprattutto se i risultati intermedi vengono materializzati (vediamo la distinzione fra materializzazione e pipelining più avanti)

Euristica per l'ottimizzazione algebrica

- Decomporre le selezioni congiuntive in successive selezioni atomiche
- Anticipare il più possibile le selezioni
- In una sequenza di selezioni, anticipare le più selettive
- Combinare prodotti cartesiani e selezioni per formare join (eventualmente riordinando gli operandi)
- Anticipare il più possibile le proiezioni (anche introducendone di nuove); peraltro questo serve soprattutto se i risultati intermedi vengono materializzati (vediamo la distinzione fra materializzazione e pipelining più avanti)

Euristica per l'ottimizzazione algebrica

- Decomporre le selezioni congiuntive in successive selezioni atomiche
- Anticipare il più possibile le selezioni
- In una sequenza di selezioni, anticipare le più selettive
- Combinare prodotti cartesiani e selezioni per formare join (eventualmente riordinando gli operandi)
- Anticipare il più possibile le proiezioni (anche introducendone di nuove)
  - peraltro questo serve soprattutto se i risultati intermedi vengono materializzati (vediamo la distinzione fra materializzazione e pipelining più avanti)

Esempio

R1(ABC), R2(DEF), R3(GHI)

```
SELECT   A , E
FROM     R1, R3, R2
WHERE    C=D AND B>100 AND F=G AND H=7 AND I>2
```

Alla lavagna

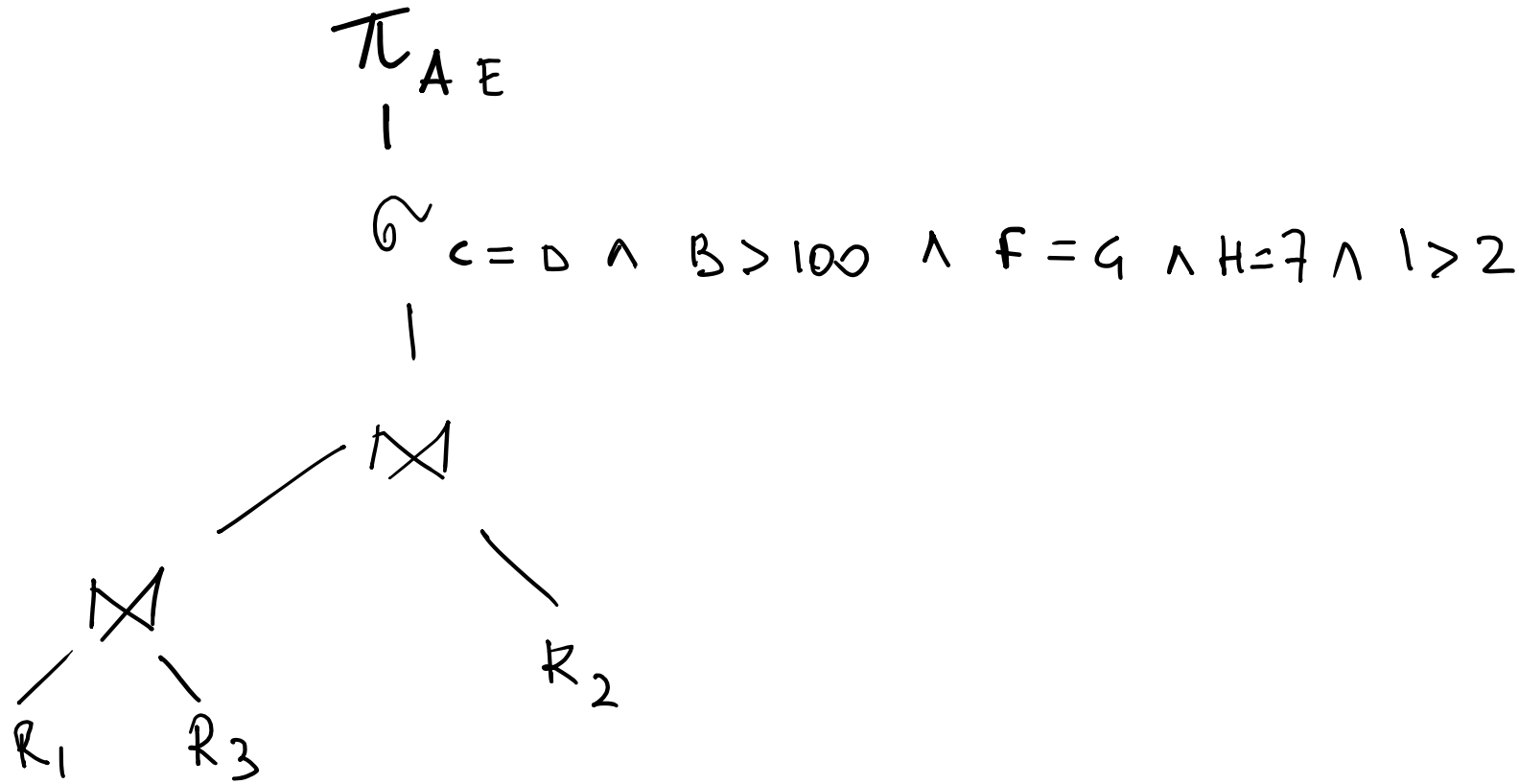
Esempio

R1(ABC), R2(DEF), R3(GHI)

SELECT **A , E**

FROM **R1, R3, R2**

WHERE **C=D AND B>100 AND F=G AND H=7 AND I>2**



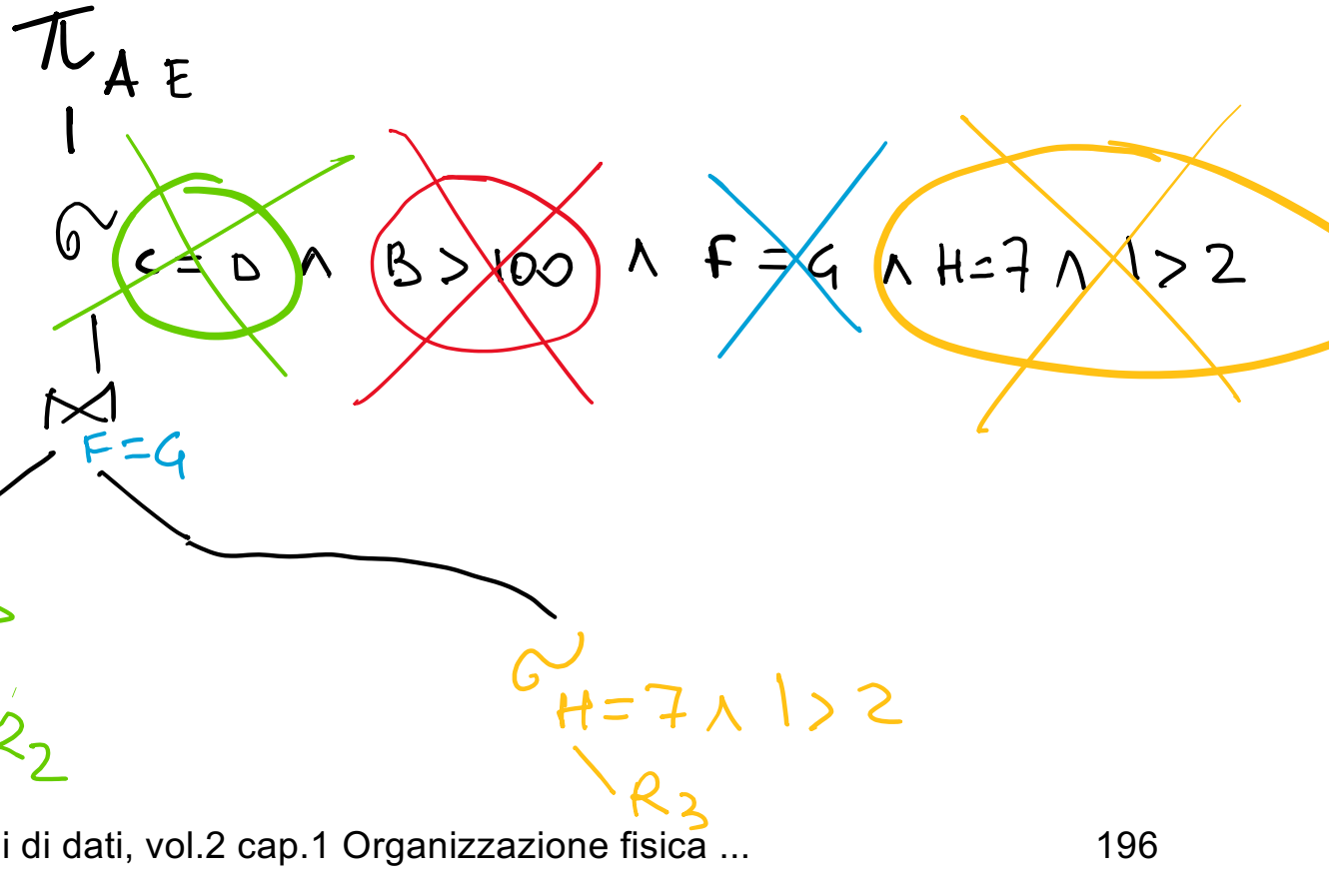
Esempio

R1(ABC), R2(DEF), R3(GHI)

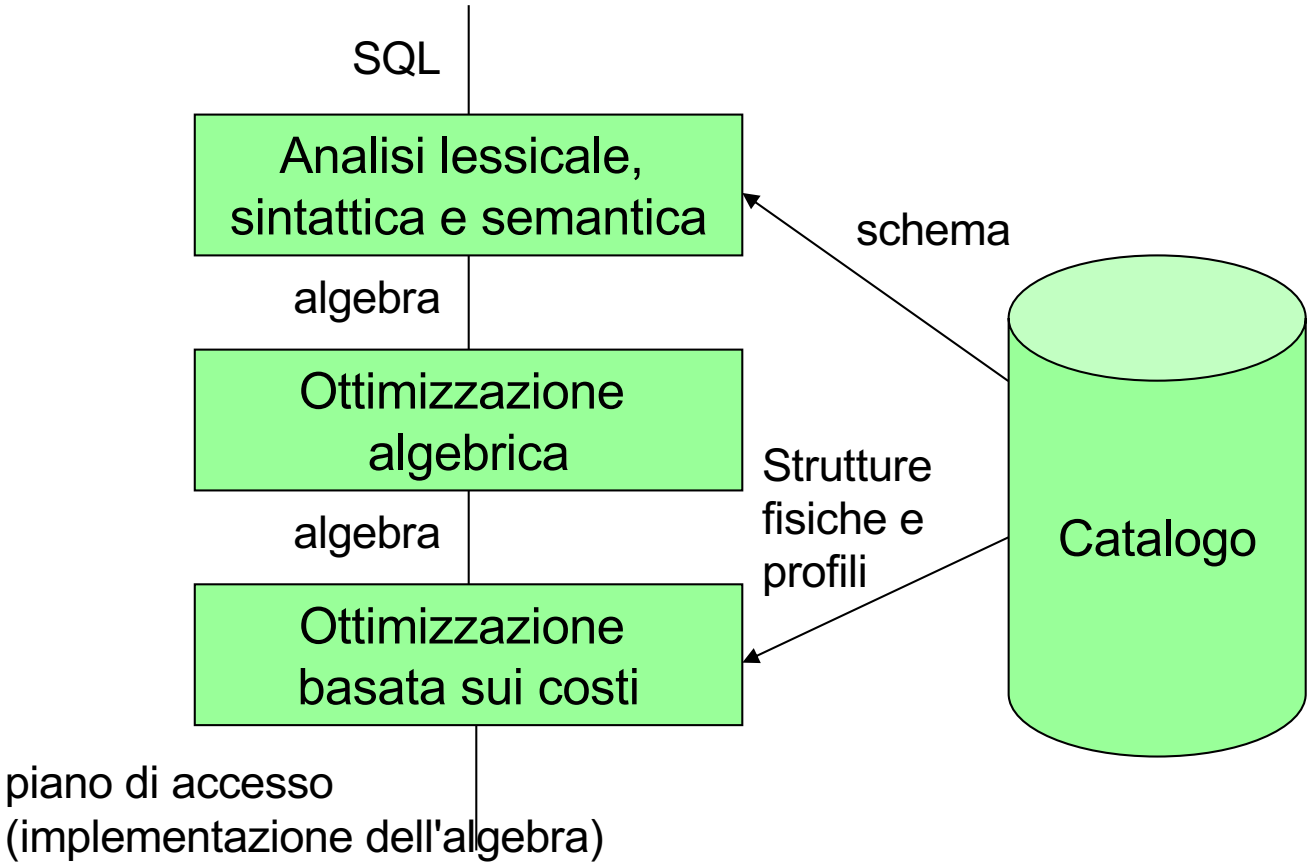
SELECT **A , E**

FROM **R1, R3, R2**

WHERE **C=D AND B>100 AND F=G AND H=7 AND I>2**



Il processo di esecuzione delle interrogazioni



Esecuzione delle operazioni

- I DBMS offrono operatori
  - associati alle strutture fisiche
  - che implementano ciascuno uno o più operatori dell'algebra
- Operatori fondamentali:
  - **scansione**
  - **accesso diretto**
- A livello più alto:
  - ordinamento
- Ancora più alto
  - join

Scansione

- Accesso sequenziale ad un file (tabella), con la possibilità di eseguire:
  - lettura completa
  - selezione (su un qualunque predicato sulle ennuple)
  - proiezione su un insieme di attributi
    - senza eliminazione di duplicati
  - inserimenti, eliminazioni, modifiche di ennuple “correnti”
- Metodi (intuitivamente):

  `open`, `next`, `read`, `modify`, `insert`, `delete`, `close`
- Costo:
  - lineare (rispetto al numero di blocchi del file)

Accesso diretto

- Può essere eseguito solo se le strutture fisiche lo permettono
  - indici
  - strutture hash

Accesso diretto basato su indice

- Utile per selezioni, cioè interrogazioni sulla "chiave" dell'indice
  - "puntuali" ($A_i = v$)
  - su intervallo ($v_1 \leq A_i \leq v_2$)
  - purché l'indice sia selettivo (esercizio visto)
- Costo: due componenti
  - profondità dell'indice, logaritmico in una base molto grande;
  - accesso ai record (ricordiamo la selettività)
- Inoltre:
  - benefici buffer (ricordiamo l'esercizio visto a suo tempo)

Selezione congiuntiva

$$\mathbf{SEL}_{\mathbf{H=7\ AND\ I=2}}\ \mathbf{(R)}$$

Selezione congiuntiva

SEL $_{H=7 \text{ AND } I=2}$ (R)

- Quanti record soddisfano **H=7**?
- Quanti record soddisfano **I=2**?
- Come procediamo, se abbiamo due indici?
  - Usare l'indice sulla condizione più selettiva e poi valutare l'altra
  - Se fossero entrambe selettive, che cosa si potrebbe fare?
    - usare entrambi gli indici, ottenendo liste di indirizzi (ai record o ai blocchi)
    - eseguire l'intersezione delle liste, come?
      - se gli indirizzi sono ai record, gli indirizzi nell'interesezione sono quelli dei record che interessano
      - se gli indirizzi sono a blocchi, allora nell'interesezione abbiamo i blocchi che potrebbero contenere i record che interessano

Selezione disgiuntiva

$$\mathbf{SEL}_{\mathbf{H=7\ OR\ I=2}}\ \mathbf{(R)}$$

- Quanti record hanno **H=7**?
- Quanti record hanno **I=2**?
- Come procediamo
- Se i campi sono entrambi selettivi e abbiamo due indici
  - possiamo usarli e poi fare l'unione dei risultati
- Se anche uno solo non è selettivo
  - allora è inutile usare gli indici, perché è comunque necessario accedere a molti record, diffusi in tutto il file, e quindi il costo non scende rispetto a quello della ricerca sequenziale

Indici su campo non chiave

- Costo (se si deve accedere ai record con il valore cercato)
  - profondità dell'indice (meno eventuali benefici dal buffer)
  - più costo dell'accesso ai record
    - indice secondario:
      - può servire un accesso per ogni record, perché sono sparpagliati
    - indice primario:
      - i record sono consecutivi (approssimativamente serve un numero di blocchi pari al numero di record trovati diviso il fattore di blocco)

Accesso diretto basato su hash

- Efficiente per interrogazioni (sulla "chiave" dell'indice)
  - "puntuali" ($A_i = v$)
  - **NON** su intervallo ($v_1 \leq A_i \leq v_2$)
- Costo:
  - approssimabile a costante
- Per predicati congiuntivi e disgiuntivi, vale lo stesso discorso fatto per gli indici

Indici e hash su più campi

- Indice su cognome e nome
  - funziona per accesso diretto su cognome?
  - funziona per accesso diretto su nome?
- Hash su cognome e nome
  - funziona per accesso diretto su cognome?
  - funziona per accesso diretto su nome?

Esecuzione delle operazioni

- I DBMS offrono operatori che implementano ciascuno uno o più operatori dell'algebra
- Operatori fondamentali:
  - scansione
  - accesso diretto
- A livello più alto:
  - **ordinamento**
- Ancora più alto
  - join

Ordinamento

- Importante, per
  - produrre risultati ordinati — ORDER BY
  - eliminare duplicati — DISTINCT
  - preparare aggregazioni — GROUP BY
  - preparare i join — vedremo
- Utilizza significativamente i buffer

Ordinamento, con buffer

- Esempio:
  - file di 1.000.000.000 di record di 100 byte ciascuno (100GB)
  - blocchi di 10KB
  - buffer disponibile di 100MB

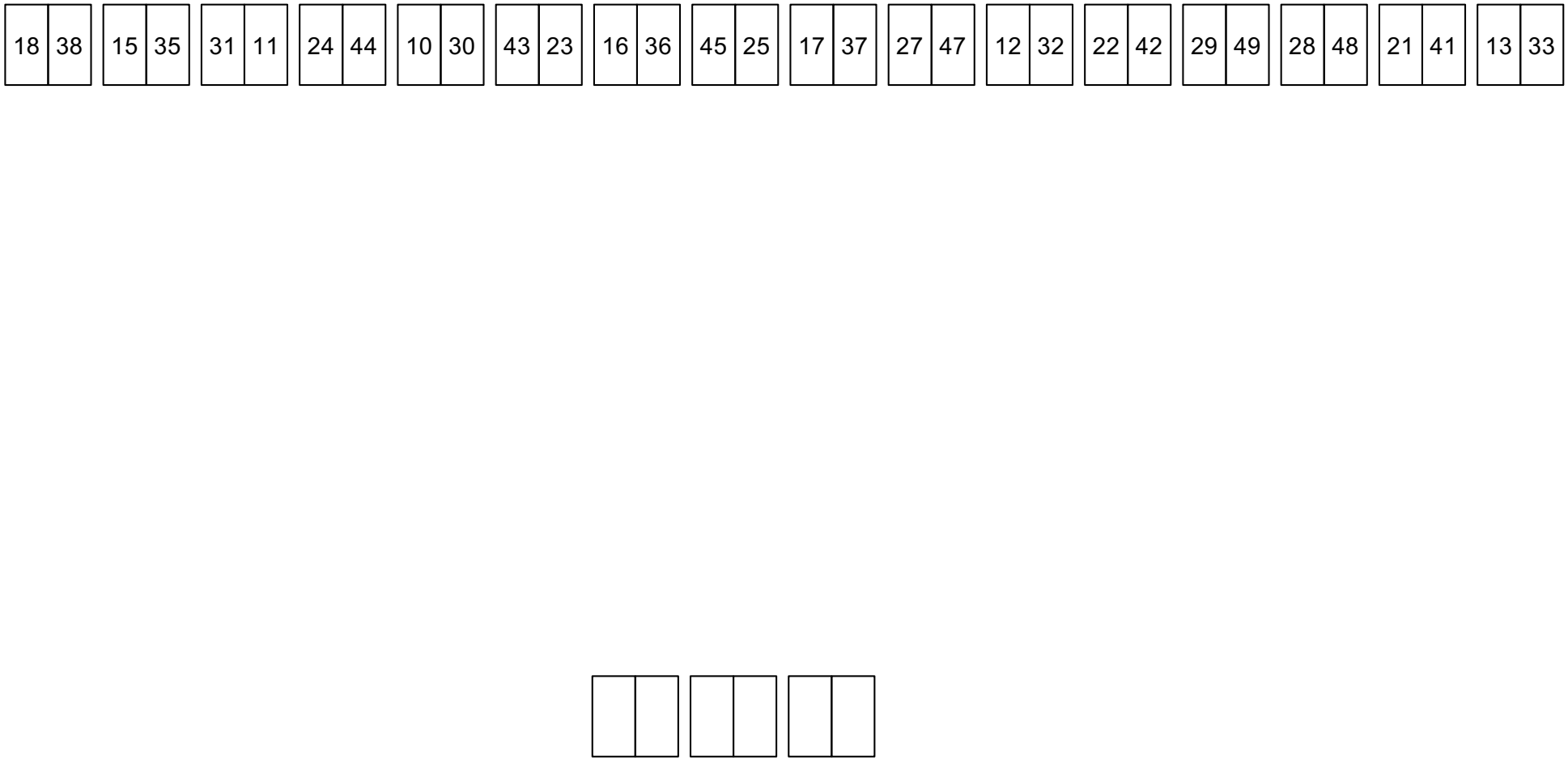
  Come possiamo procedere?
  - merge-sort …

Tradizionale ordinamento di file

- Merge-sort "esterno" (in memoria secondaria e "poca" memoria principale):
    - divido il file a metà
    - ordino separatamente ciascuna metà
        - se necessario procedo ricorsivamente
        - se la porzione è piccola (ad esempio un elemento) opero in memoria
    - fondo le due metà ("merge")

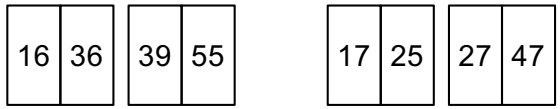
- esempio alla lavagna (16 blocchi e 3 buffer)
- poi vedremo con più pagine di buffer

Merge sort esterno con tre buffer

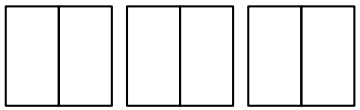


Premessa:
merge sort passo fondamentale, merge

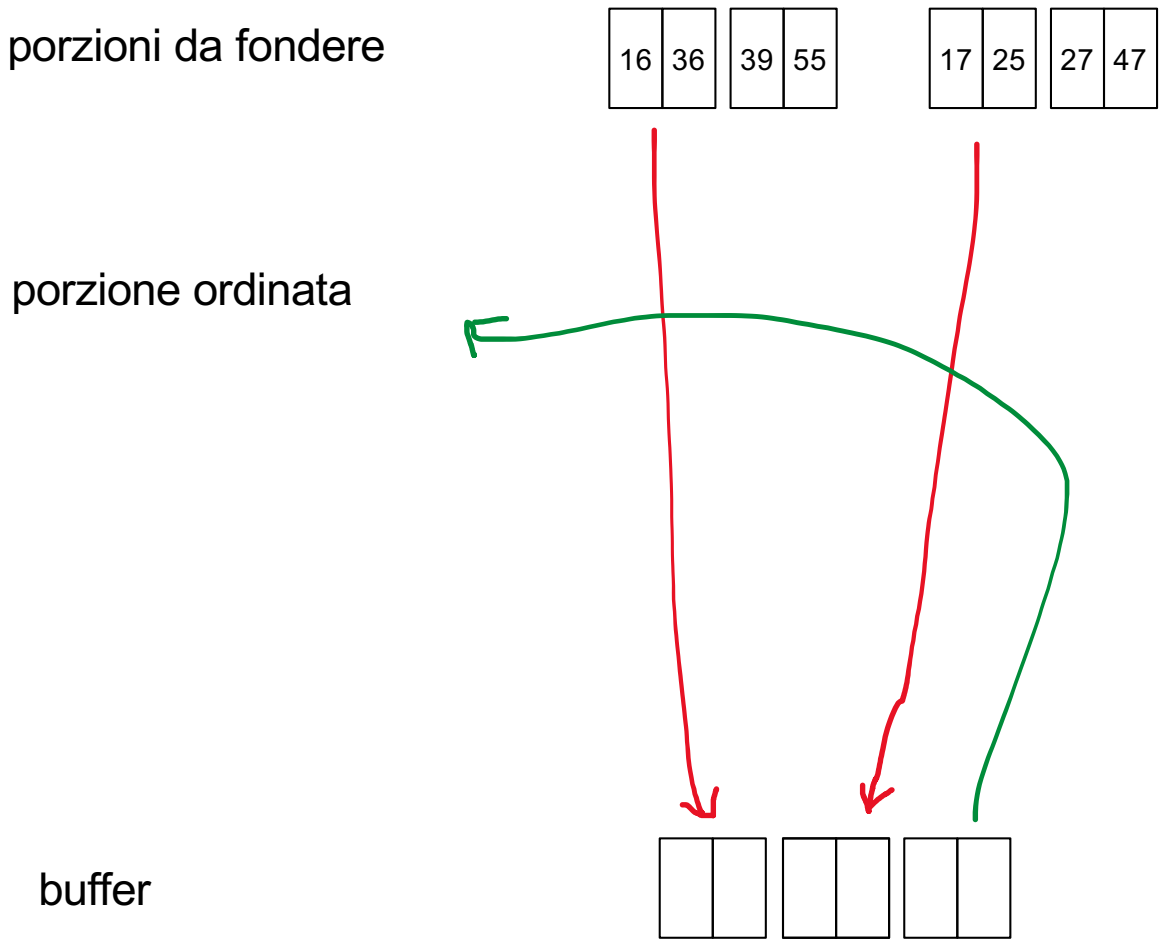
- Due piccole porzioni ordinate, le fondiamo



- Tre pagine di buffer a disposizione

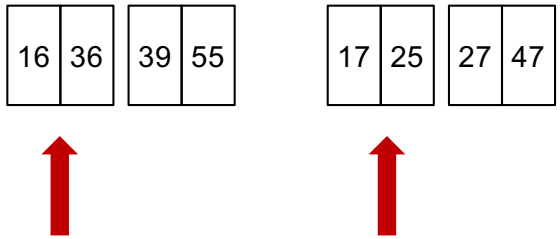


Merge sort passo fondamentale



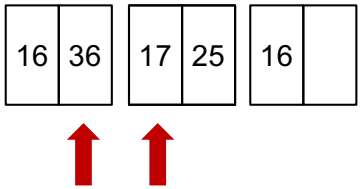
Merge sort passo fondamentale

porzioni da fondere



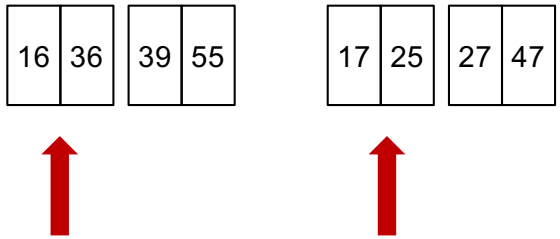
porzione ordinata

buffer



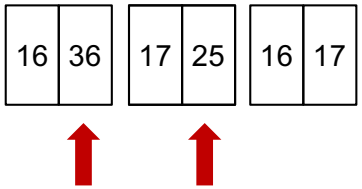
Merge sort passo fondamentale

porzioni da fondere



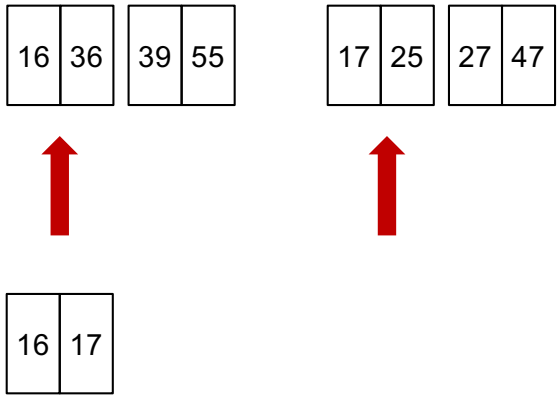
porzione ordinata

buffer



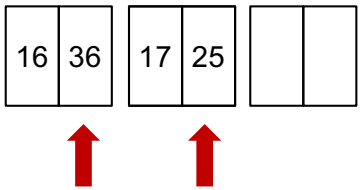
Merge sort passo fondamentale

porzioni da fondere

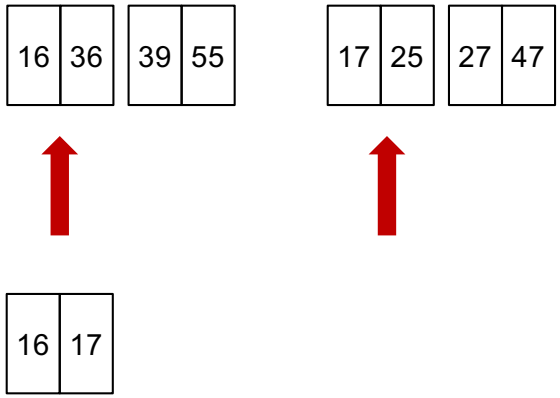


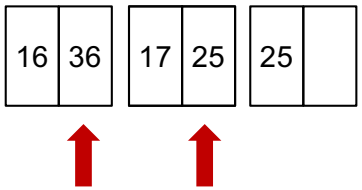
porzione ordinata

buffer

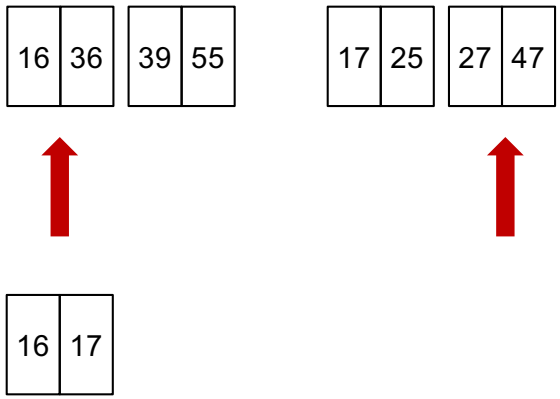


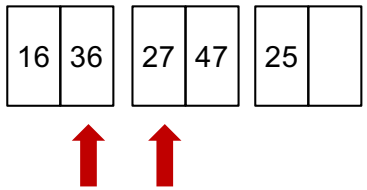
Merge sort passo fondamentale



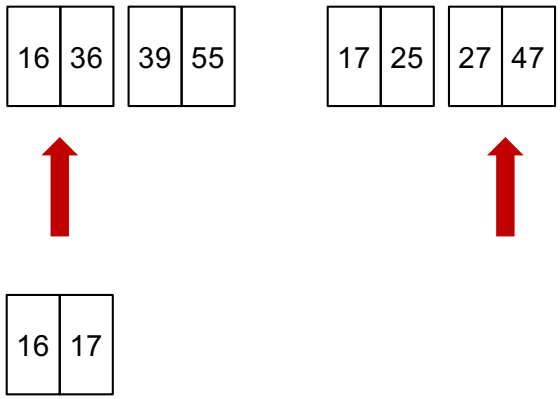


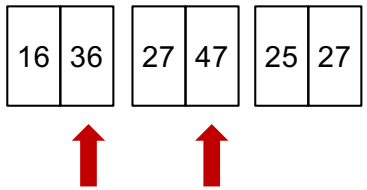
Merge sort passo fondamentale



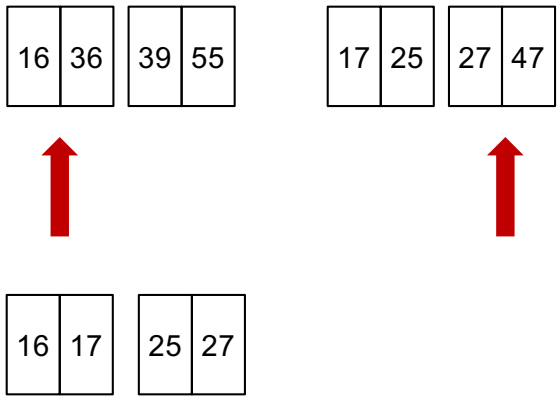


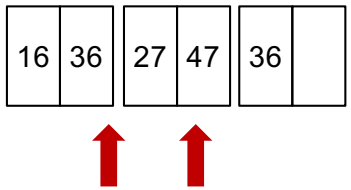
Merge sort passo fondamentale



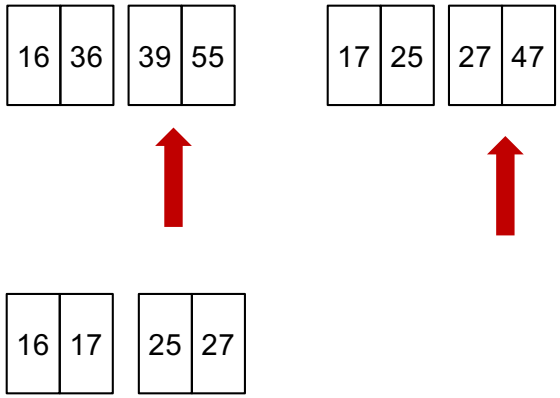


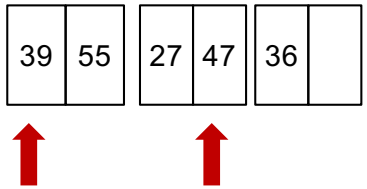
Merge sort passo fondamentale





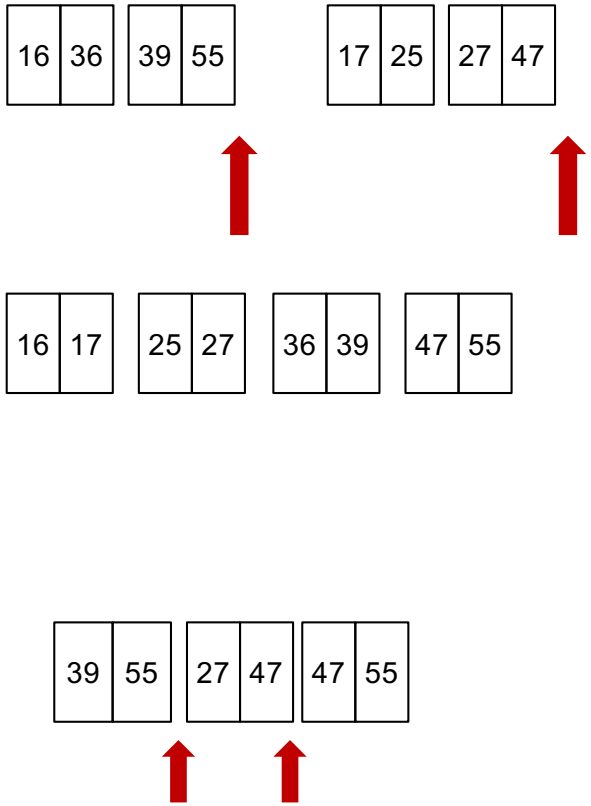
Merge sort passo fondamentale



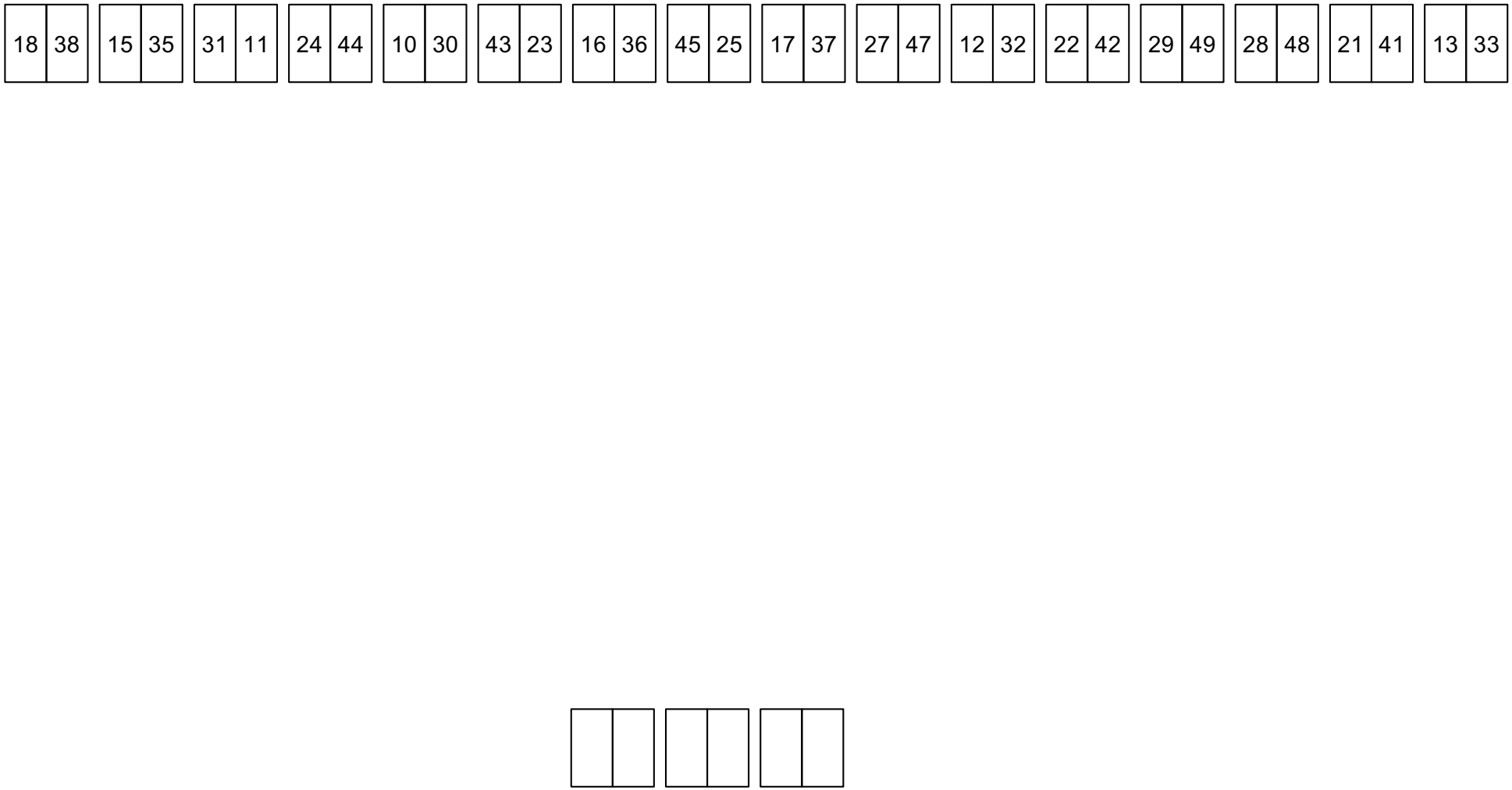


Merge sort passo fondamentale

- E così via fino alla scansione completa delle due porzioni, alla fine:



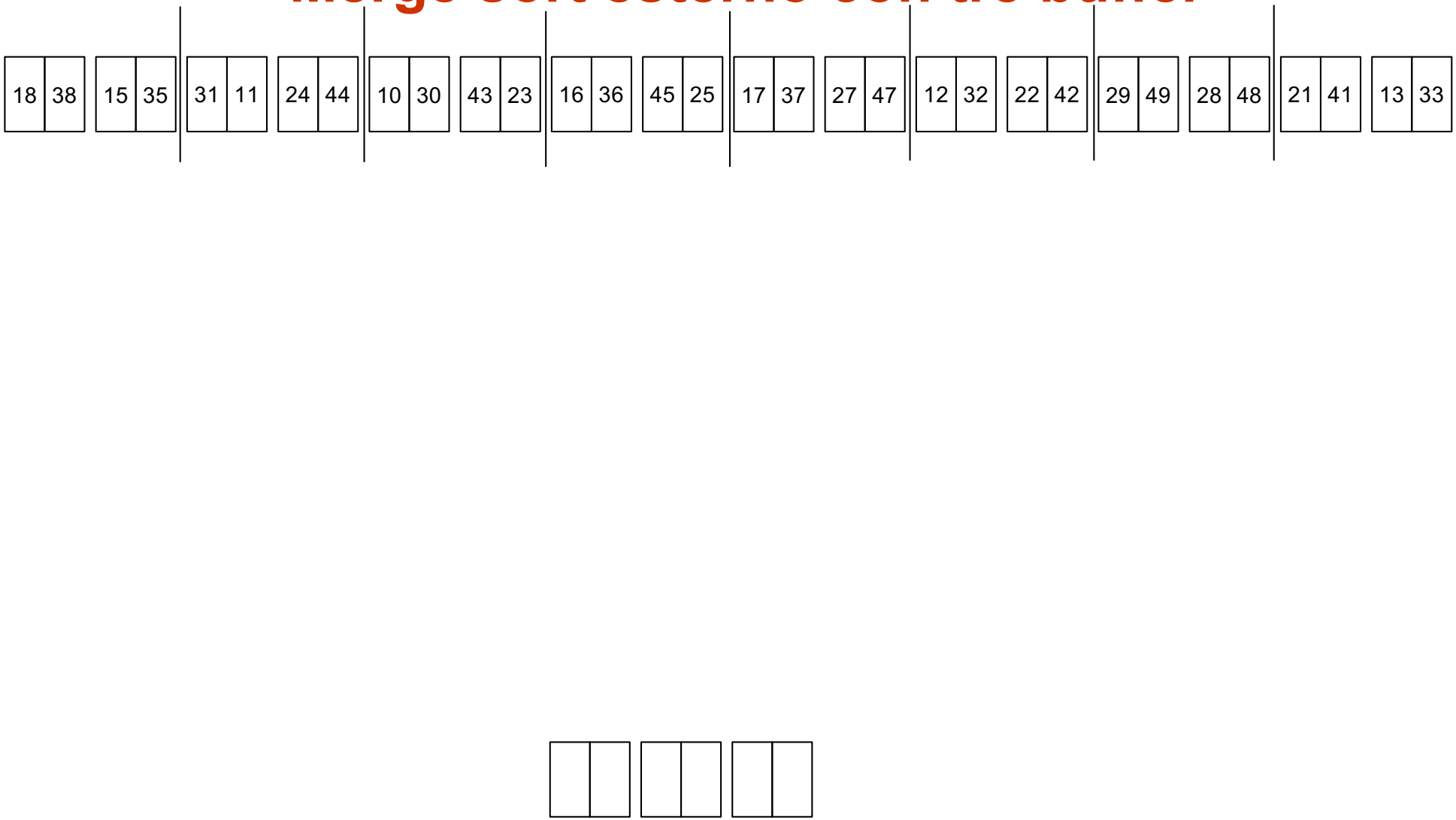
Merge sort esterno con tre buffer



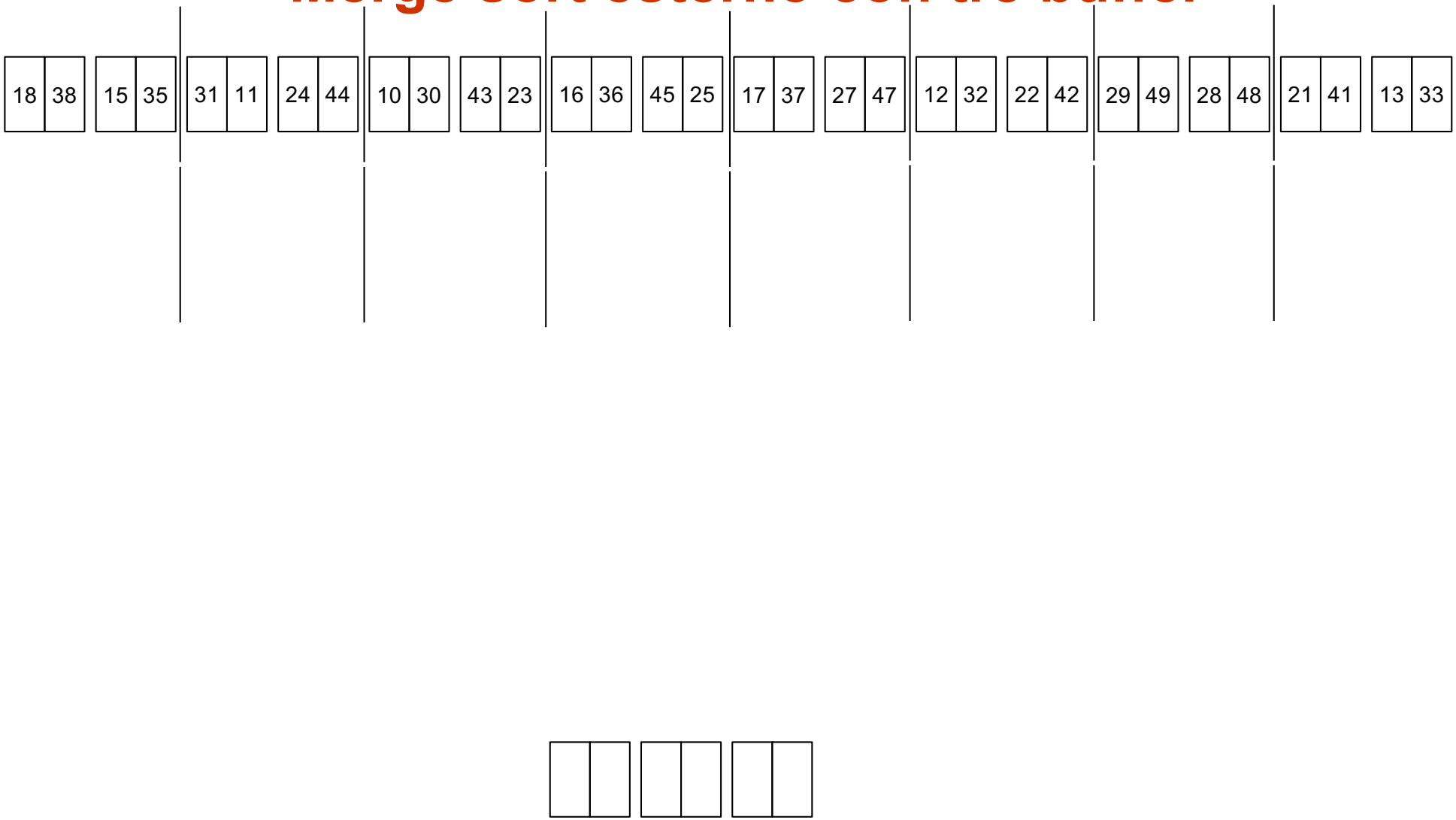
Merge sort esterno con tre buffer

- Torniamo all'algoritmo complessivo
- Nota bene:
  - il merge sort viene di solito presentato "top-down"
    - divido a metà, ordino ciascuna metà e poi fondo
  - qui conviene vederlo in modo "bottom-up"
    - ordiniamo porzioni via via più lunghe, fino ad arrivare all'intero file

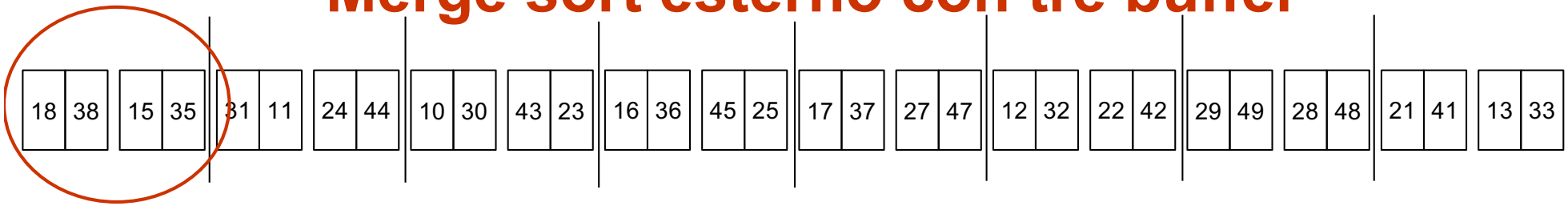
Merge sort esterno con tre buffer

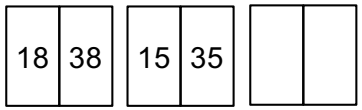


Merge sort esterno con tre buffer

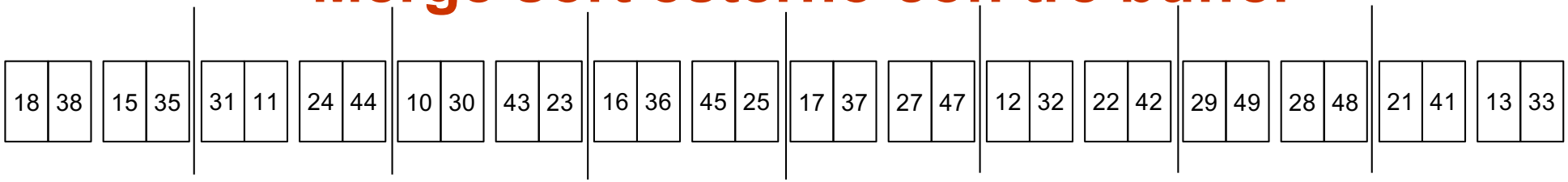


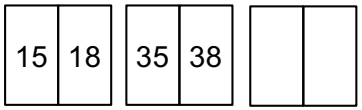
Merge sort esterno con tre buffer



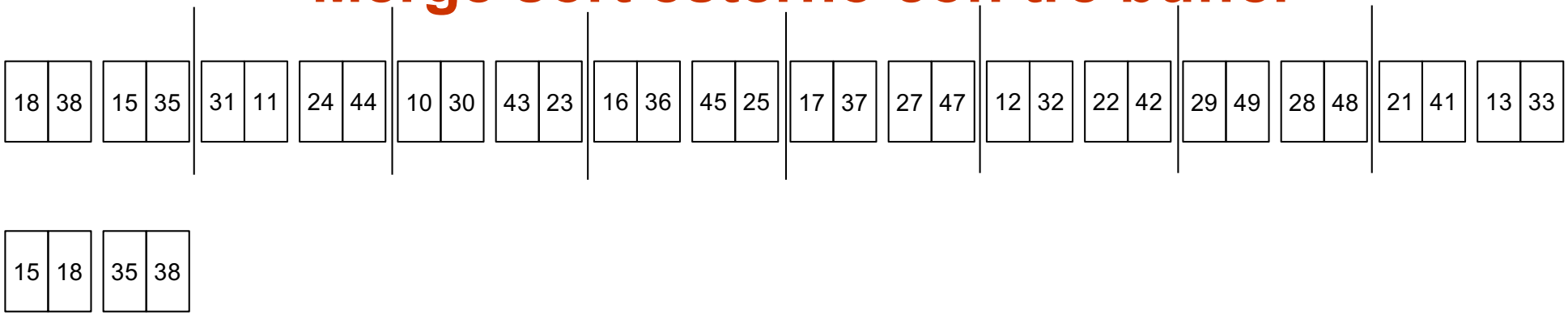


Merge sort esterno con tre buffer

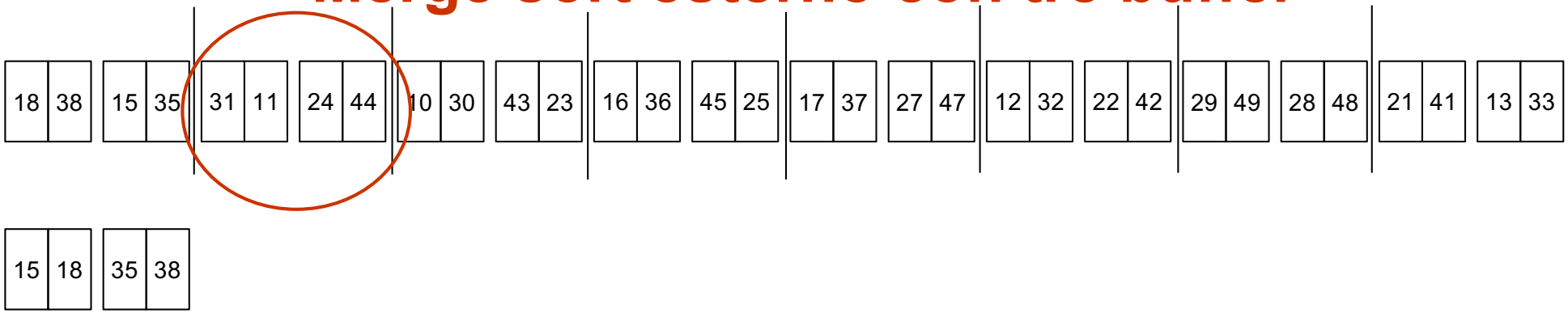


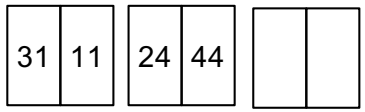


Merge sort esterno con tre buffer

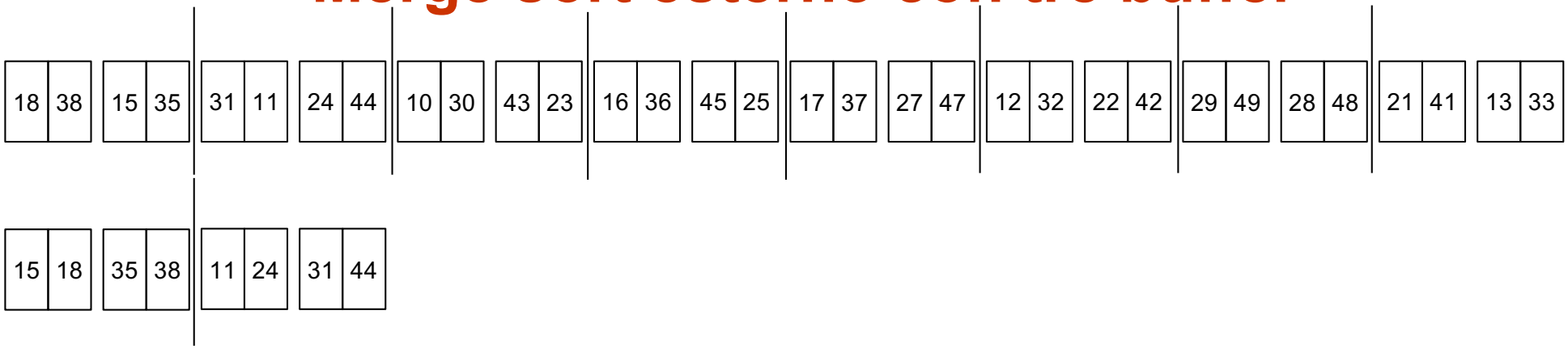


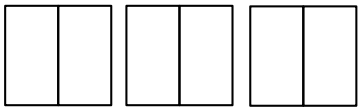
Merge sort esterno con tre buffer



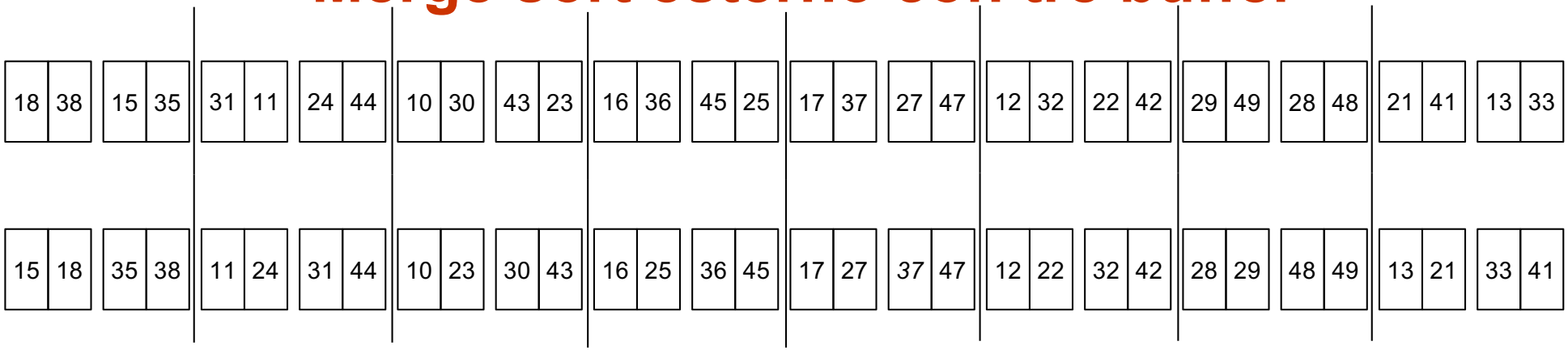


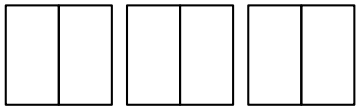
Merge sort esterno con tre buffer



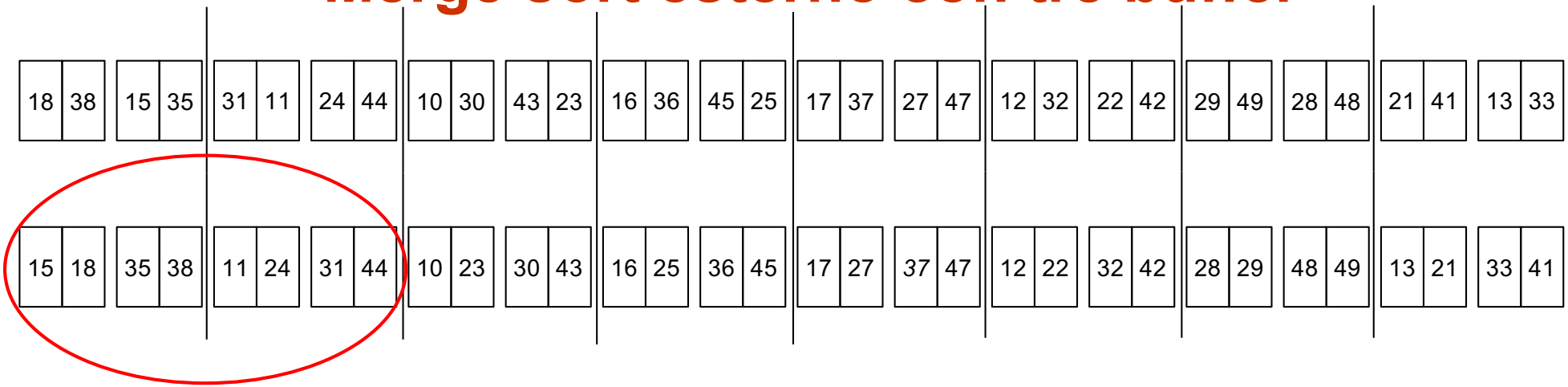


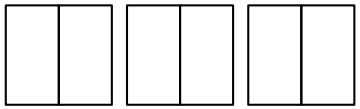
Merge sort esterno con tre buffer



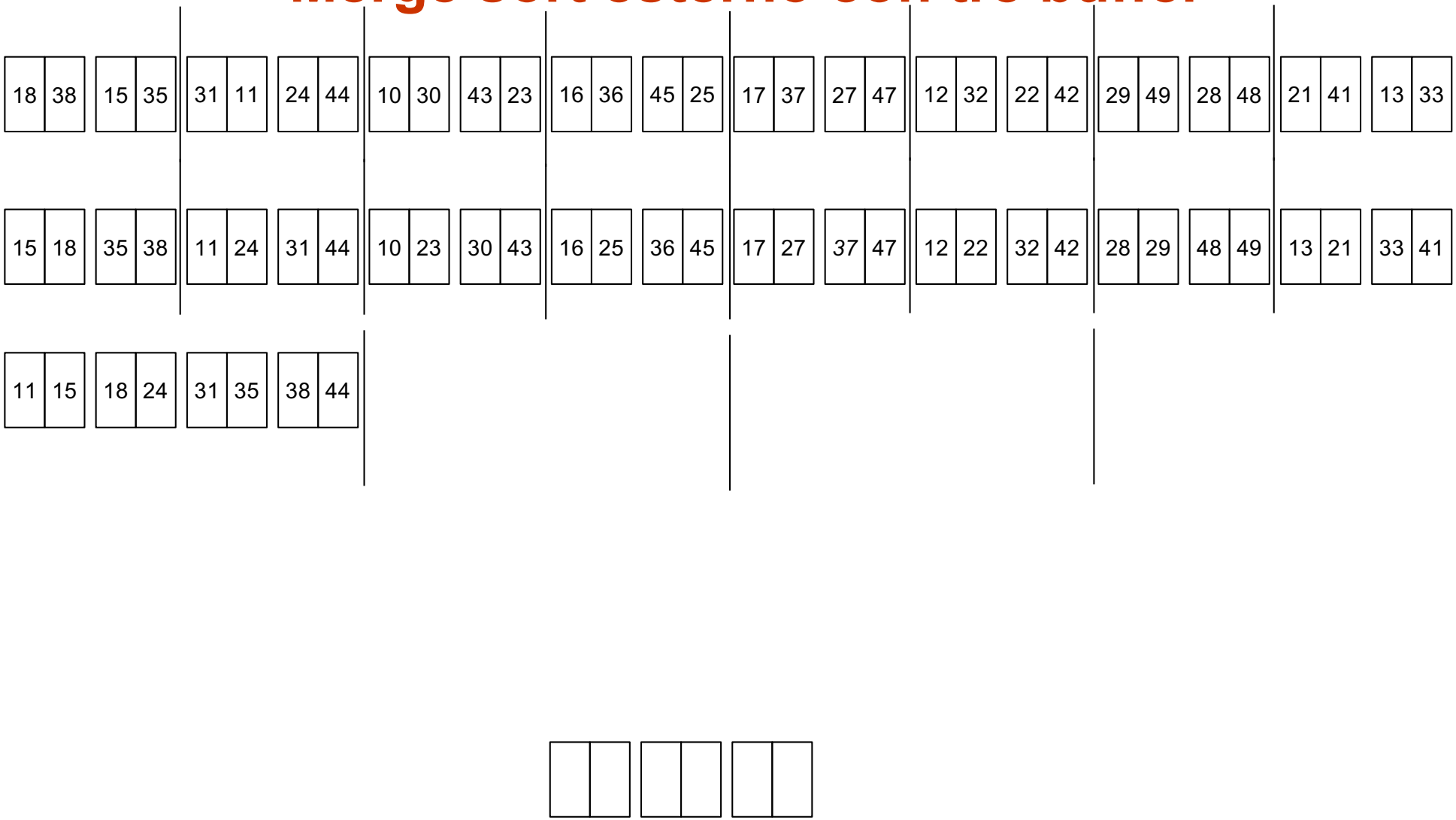


Merge sort esterno con tre buffer





Merge sort esterno con tre buffer



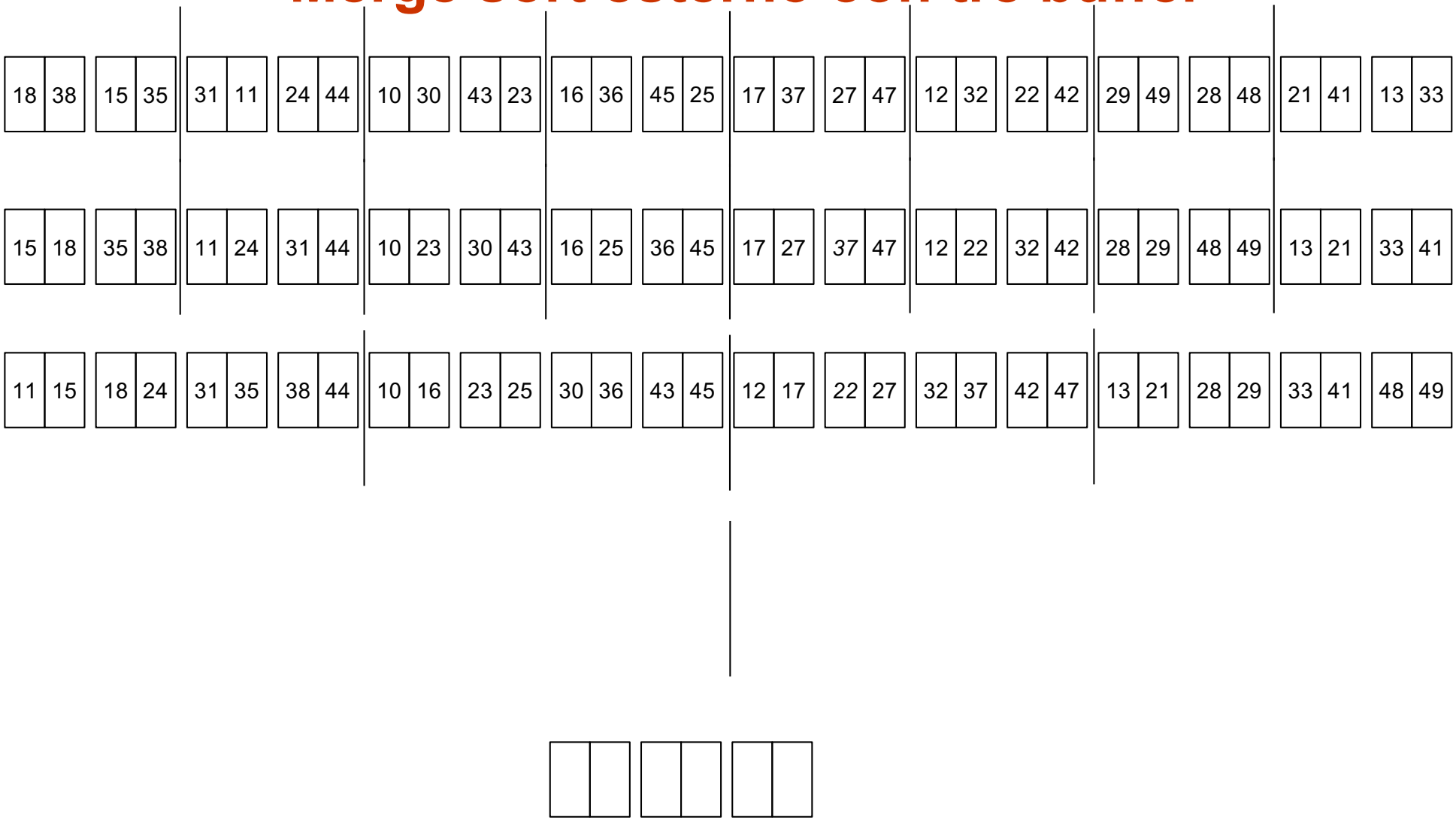
Merge sort esterno con tre buffer

18	38	15	35	31	11	24	44	10	30	43	23	16	36	45	25	17	37	27	47	12	32	22	42	29	49	28	48	21	41	13	33

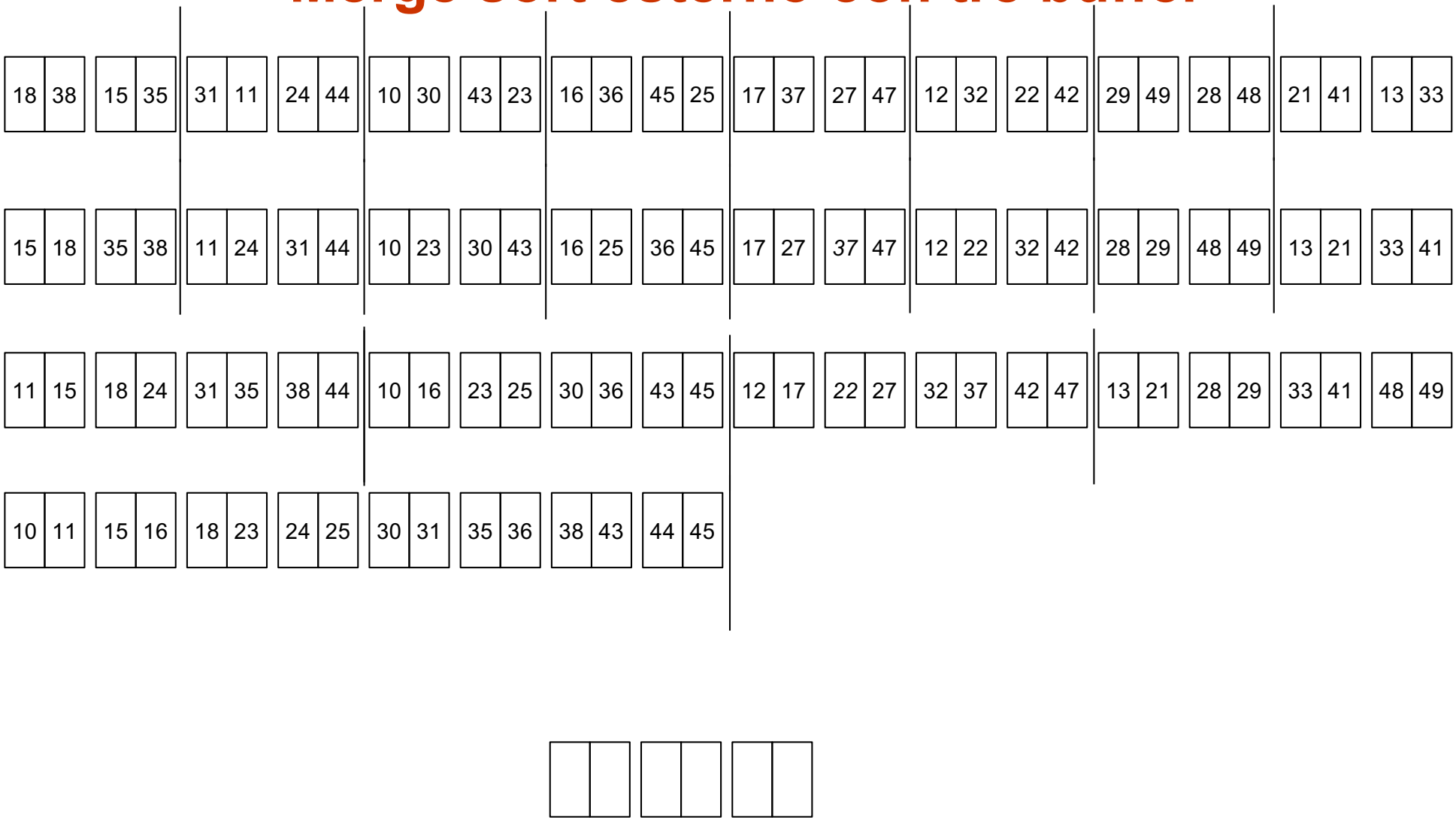
15	18	35	38	11	24	31	44	10	23	30	43	16	25	36	45	17	27	*37*	47	12	22	32	42	28	29	48	49	13	21	33	41

11	15	18	24	31	35	38	44	10	16	23	25	30	36	43	45	12	17	*22*	27	32	37	42	47	13	21	28	29	33	41	48	49

Merge sort esterno con tre buffer

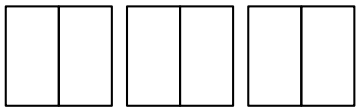


Merge sort esterno con tre buffer



Merge sort esterno con tre buffer

18	38	15	35	31	11	24	44	10	30	43	23	16	36	45	25	17	37	27	47	12	32	22	42	29	49	28	48	21	41	13	33
15	18	35	38	11	24	31	44	10	23	30	43	16	25	36	45	17	27	*37*	47	12	22	32	42	28	29	48	49	13	21	33	41
11	15	18	24	31	35	38	44	10	16	23	25	30	36	43	45	12	17	*22*	27	32	37	42	47	13	21	28	29	33	41	48	49
10	11	15	16	18	23	24	25	30	31	35	36	38	43	44	45	12	13	*17*	21	22	27	28	29	32	33	37	41	42	47	48	49



Merge sort esterno con tre buffer

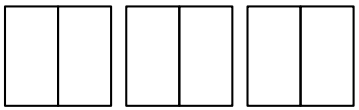
18	38	15	35	31	11	24	44	10	30	43	23	16	36	45	25	17	37	27	47	12	32	22	42	29	49	28	48	21	41	13	33

15	18	35	38	11	24	31	44	10	23	30	43	16	25	36	45	17	27	*37*	47	12	22	32	42	28	29	48	49	13	21	33	41

11	15	18	24	31	35	38	44	10	16	23	25	30	36	43	45	12	17	*22*	27	32	37	42	47	13	21	28	29	33	41	48	49

10	11	15	16	18	23	24	25	30	31	35	36	38	43	44	45	12	13	*17*	21	22	27	28	29	32	33	37	41	42	47	48	49

10	11	12	13	15	16	17	18	21	22	23	24	25	27	28	29	30	31	*32*	33	35	36	37	38	41	42	43	44	45	47	48	49



Tradizionale ordinamento di file

- Merge-sort "esterno" (con memoria secondaria e "poca" memoria principale), file di N blocchi:
  - dall'esempio alla lavagna (16 blocchi e 3 buffer), costo:
    - approssimativamente, $\log_2 N$ passi di merge, ognuno dei quali ha un costo pari a $2 \times N$ (si legge e scrive l'intero file); costo complessivo:

$$2 \times N \times \log_2 N$$

    - con 16 blocchi, $\log_2 16 = 4$, abbiamo letto e scritto 4 volte l'intero file:
      - 2 x 16 x 4 = 128

Merge sort, con buffer grandi

- Come abbiamo visto, il merge-sort "esterno" richiede:
  - $\log_2 N$ passi di merge,
  - ognuno di costo pari a $2 \times N$
- Costo complessivo: $2 \times N \times \log_2 N$
- Se abbiamo molta memoria, come possiamo migliorare?
  - abbiamo due termini, quale o quali possiamo ridurre
    - $2 \times N$? Costo del singolo passo
    - $\log_2 N$? Numero di passi

Merge sort, con buffer grandi

- Che cosa possiamo ridurre?
  - $2 \times N$: costo del merge
  - $\log_2 N$: numero di passi di merge
- Se abbiamo molta memoria,
  - non possiamo ridurre il costo del merge
  - possiamo il numero di passi di merge
- Come?
  - inizialmente, invece di ordinare singoli blocchi, ordiniamo porzioni di file ordinate (dette run) che entrano in memoria
  - poi, invece di fondere due porzioni, ne fondiamo (come estremo, forse non praticabile) tante quante sono le pagine del buffer (o quasi); in pratica, questo porta quasi sempre a un solo passo di merge o al massimo a due

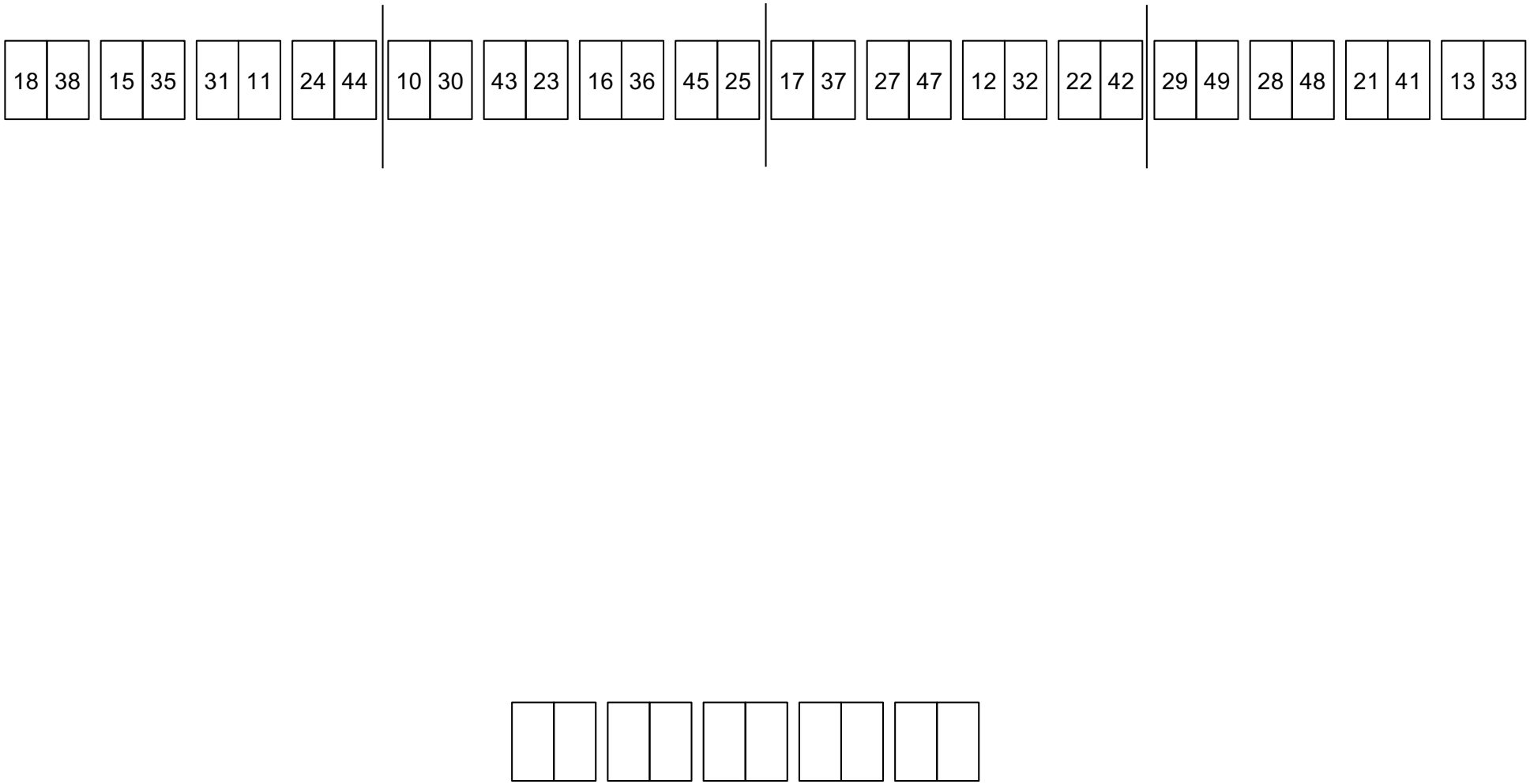
Esempio alla lavagna con sedici blocchi

- Il merge-sort tradizionale a due vie richiede quattro passi:
  - passo iniziale più tre di merge
- Con quattro vie (cinque buffer) ne bastano due:
  - prima si ordinano porzioni di quattro blocchi (potrebbero essere cinque, ma semplifichiamo i conti)
  - poi si fa una fusione di quattro porzioni

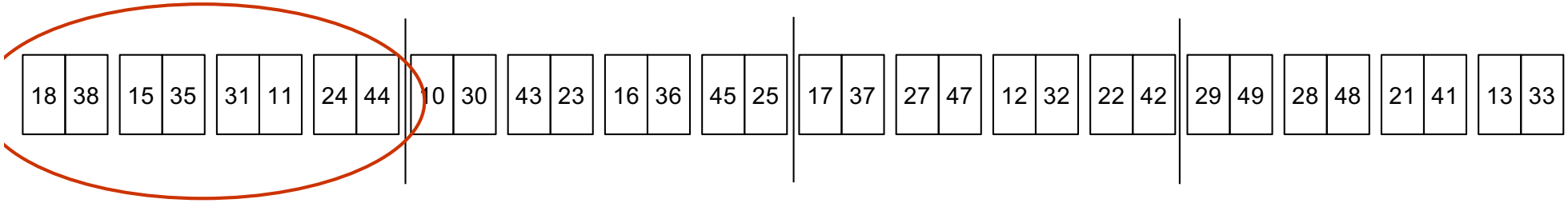
- Se ho quattro pacchetti di compiti ciascuno ordinato, come ottengo un pacco ordinato?

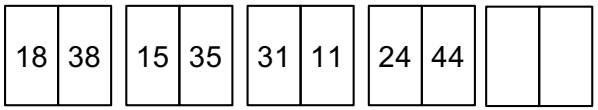
- Torniamo alla lavagna

Merge sort esterno con cinque buffer

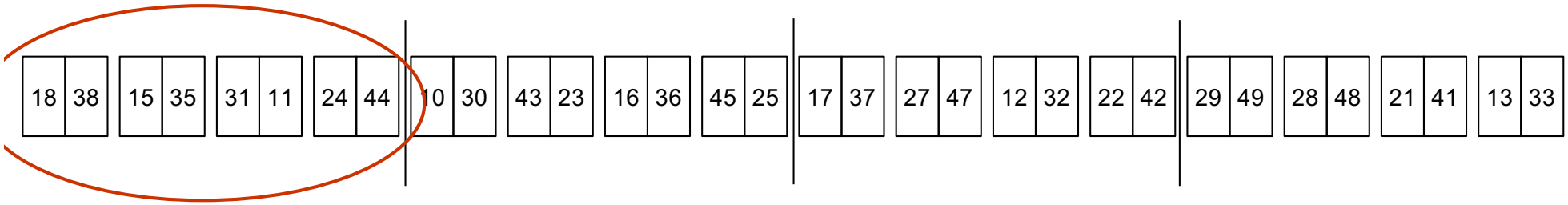


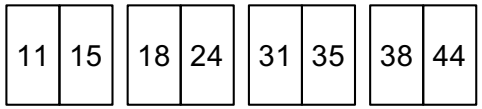
Primo passo: ordino gruppi di quattro blocchi

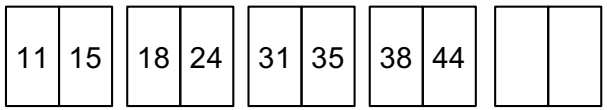




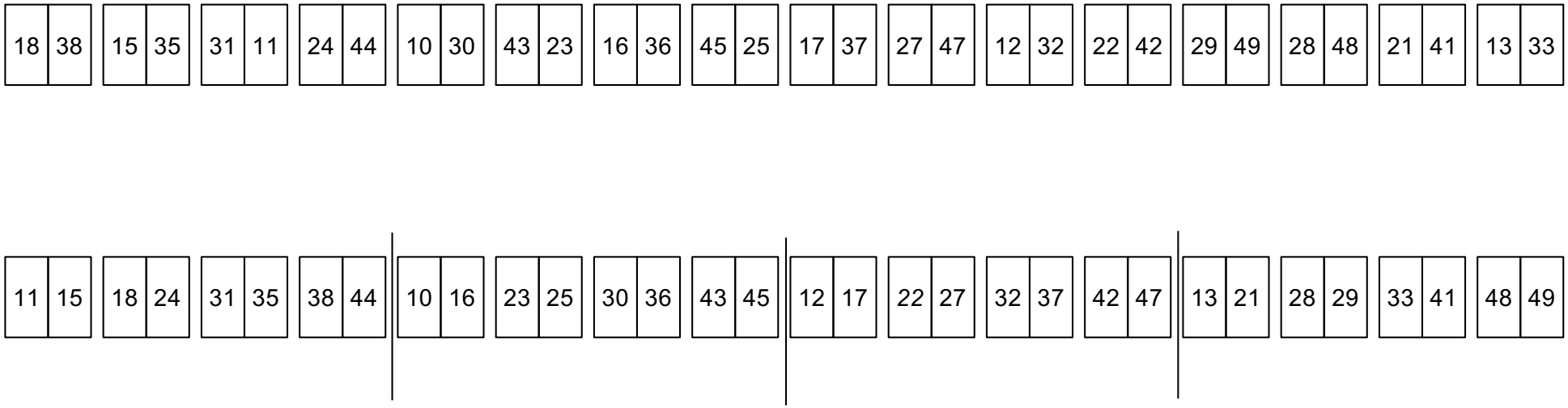
Primo passo: ordino gruppi di quattro blocchi



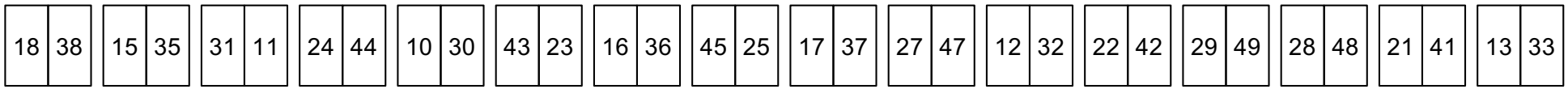


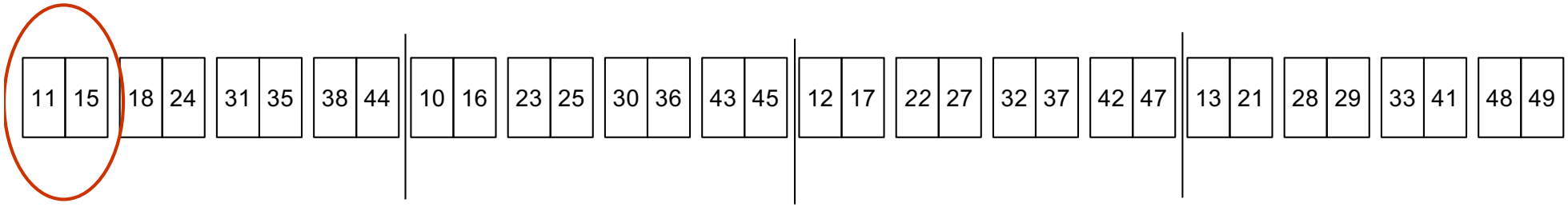


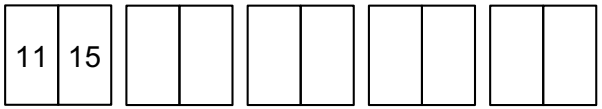
Primo passo: ordino gruppi di quattro blocchi



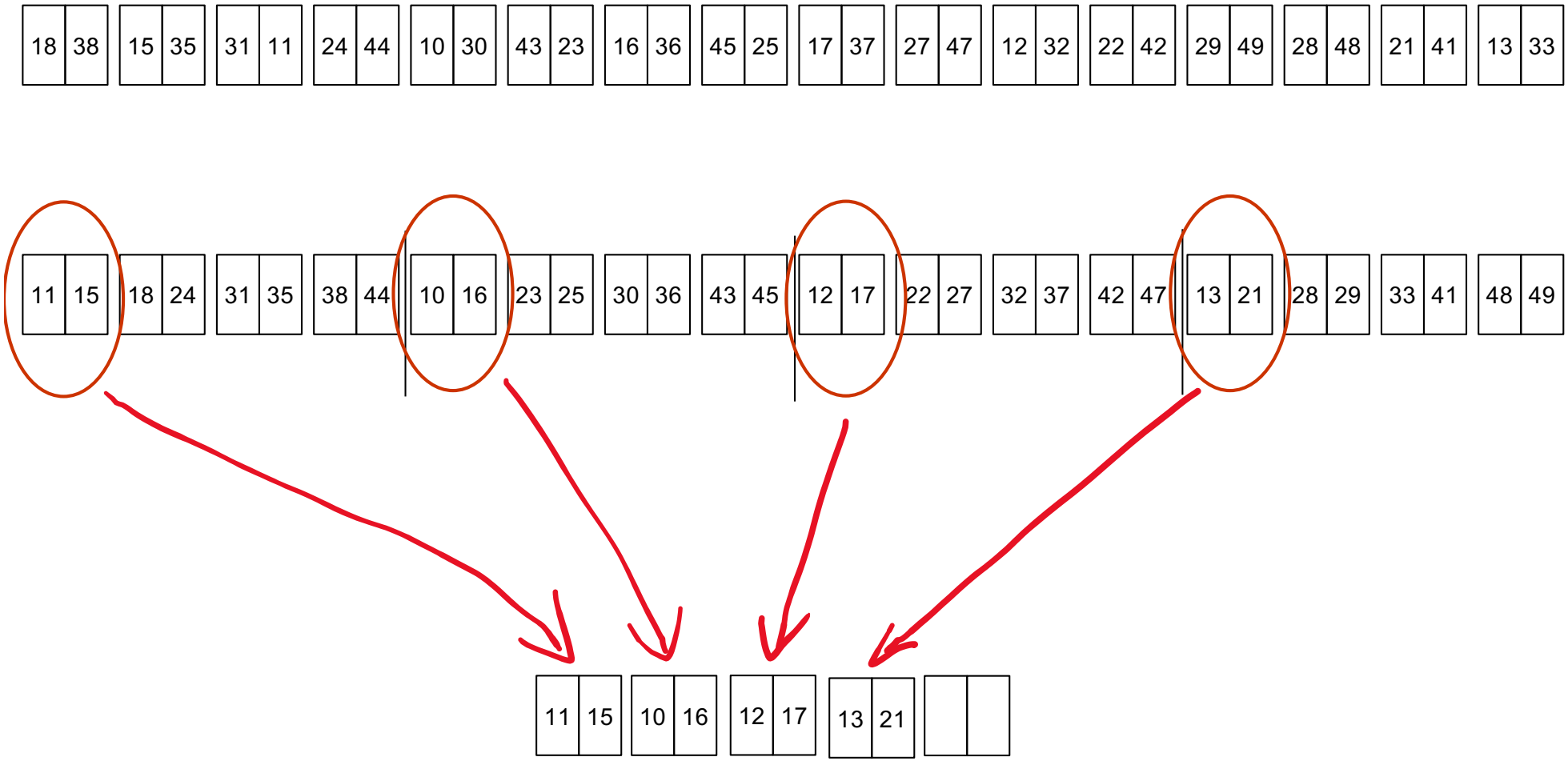
Secondo passo: merge a quattro vie



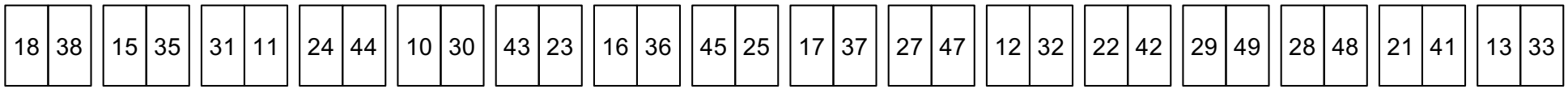


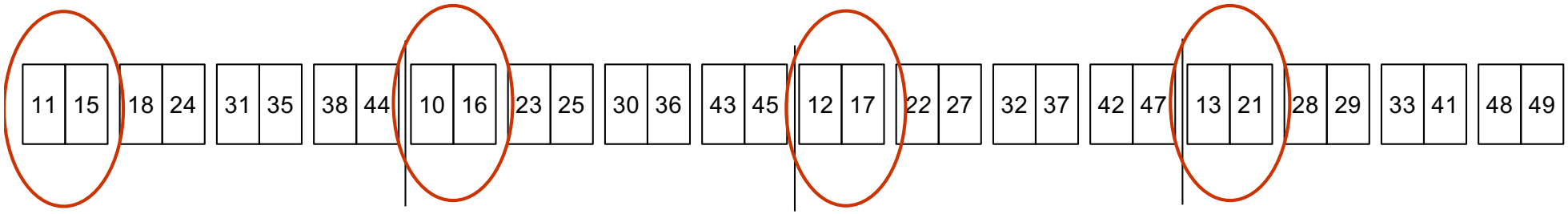


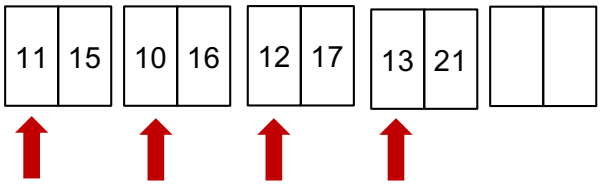
Secondo passo: merge a quattro vie



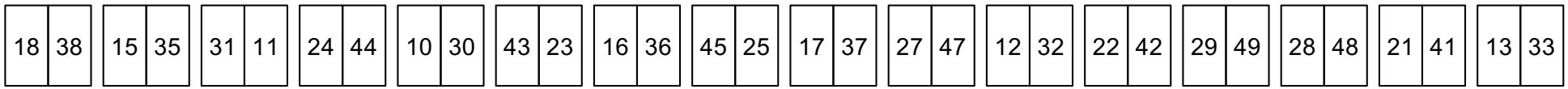
Secondo passo: merge a quattro vie

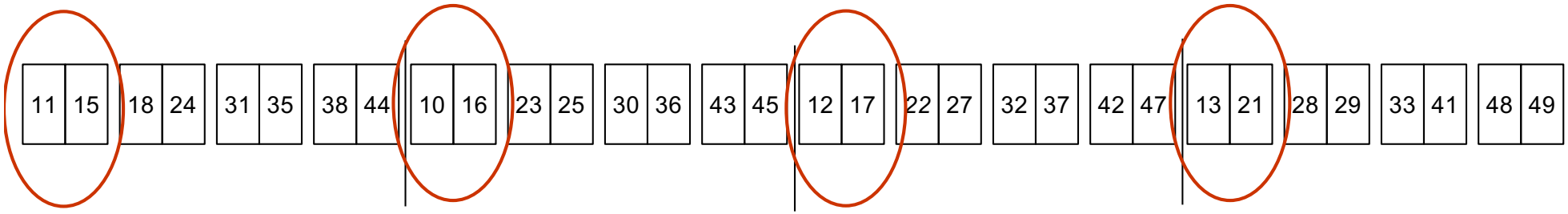


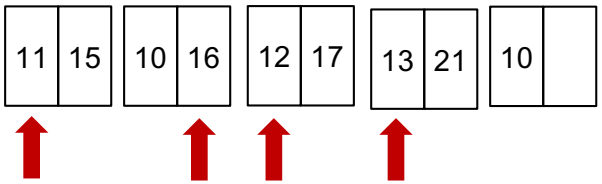




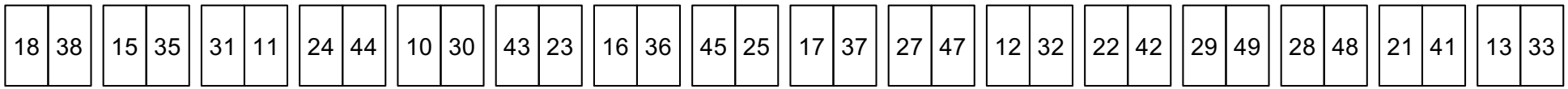
Secondo passo: merge a quattro vie

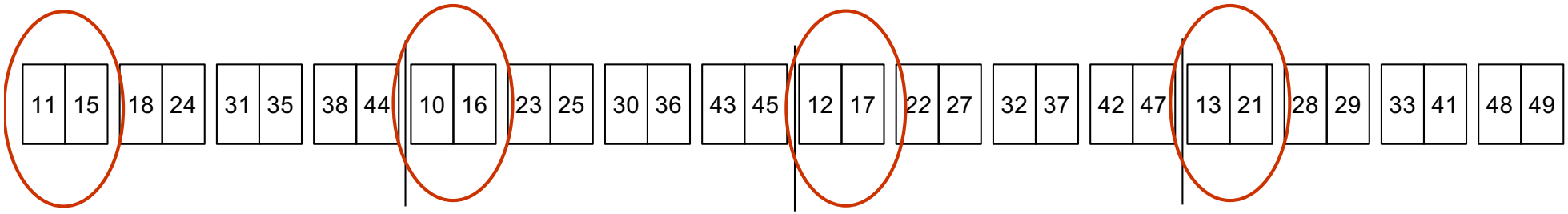


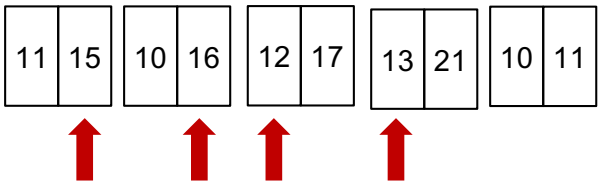




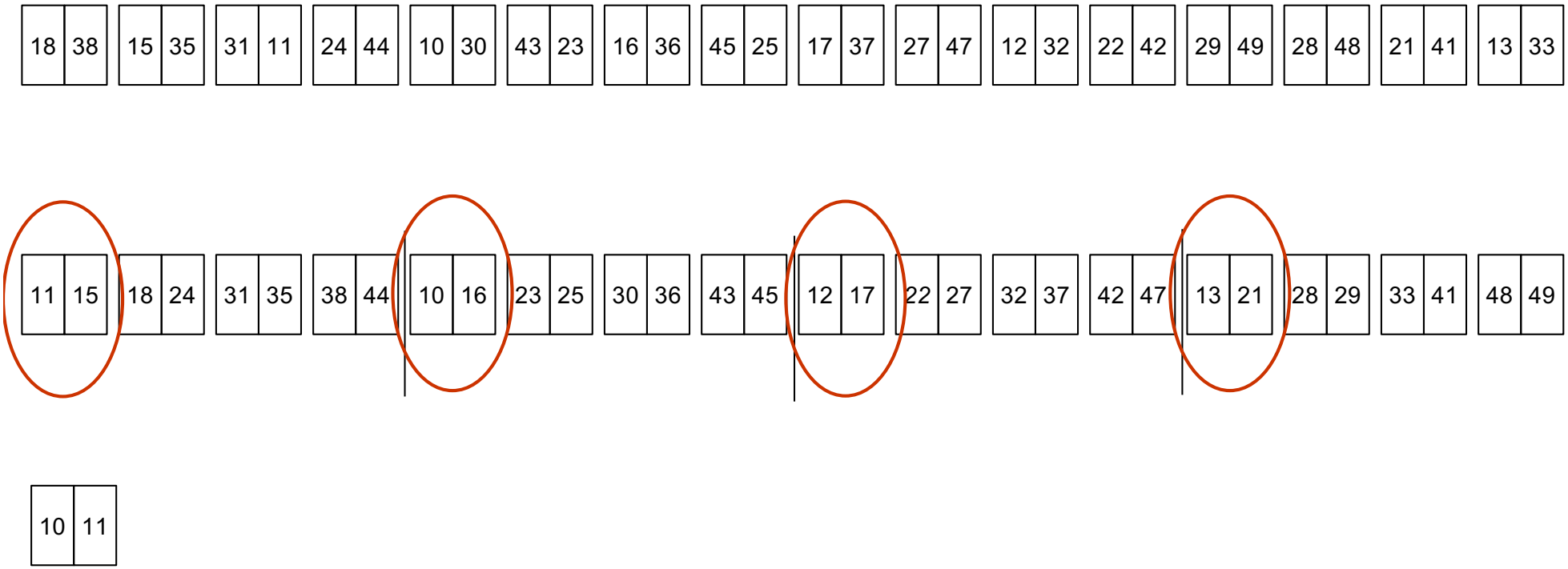
Secondo passo: merge a quattro vie

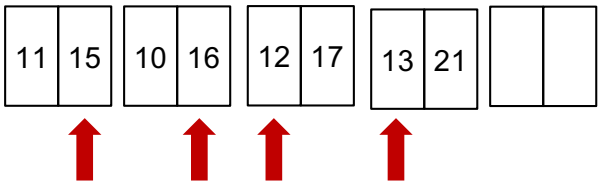




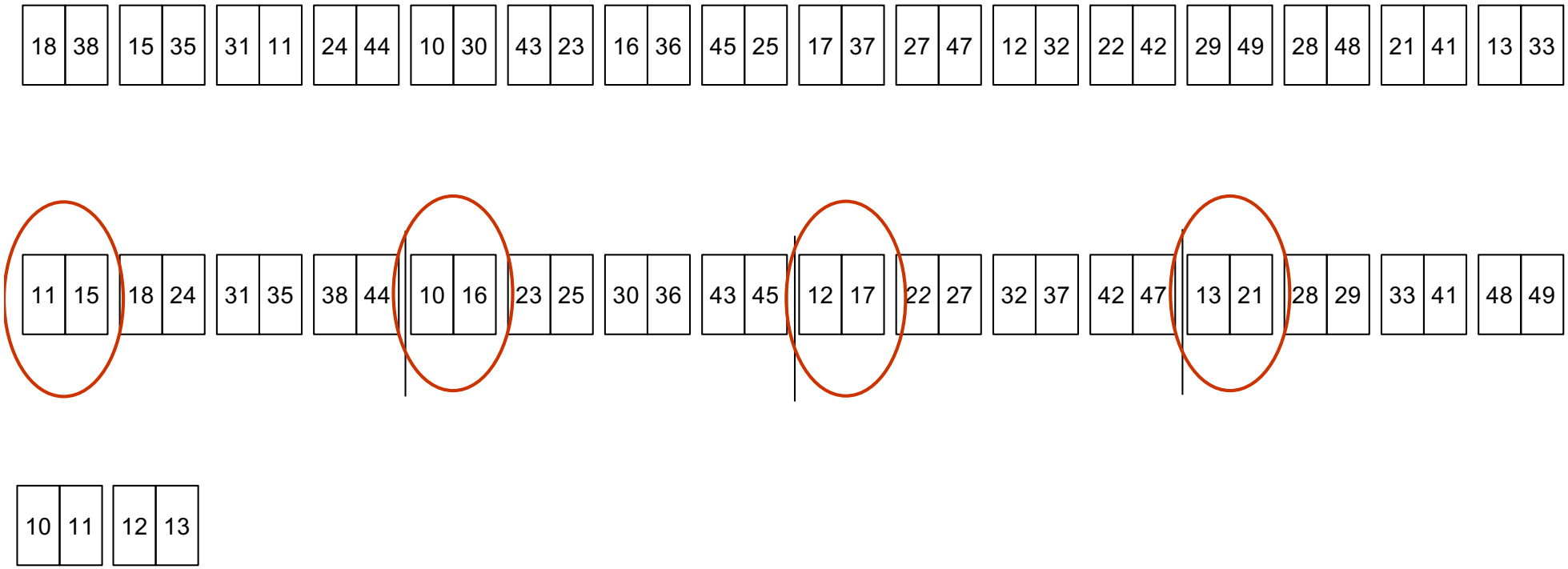


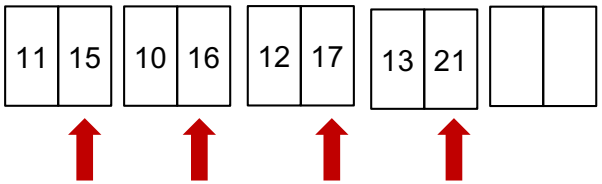
Secondo passo: merge a quattro vie



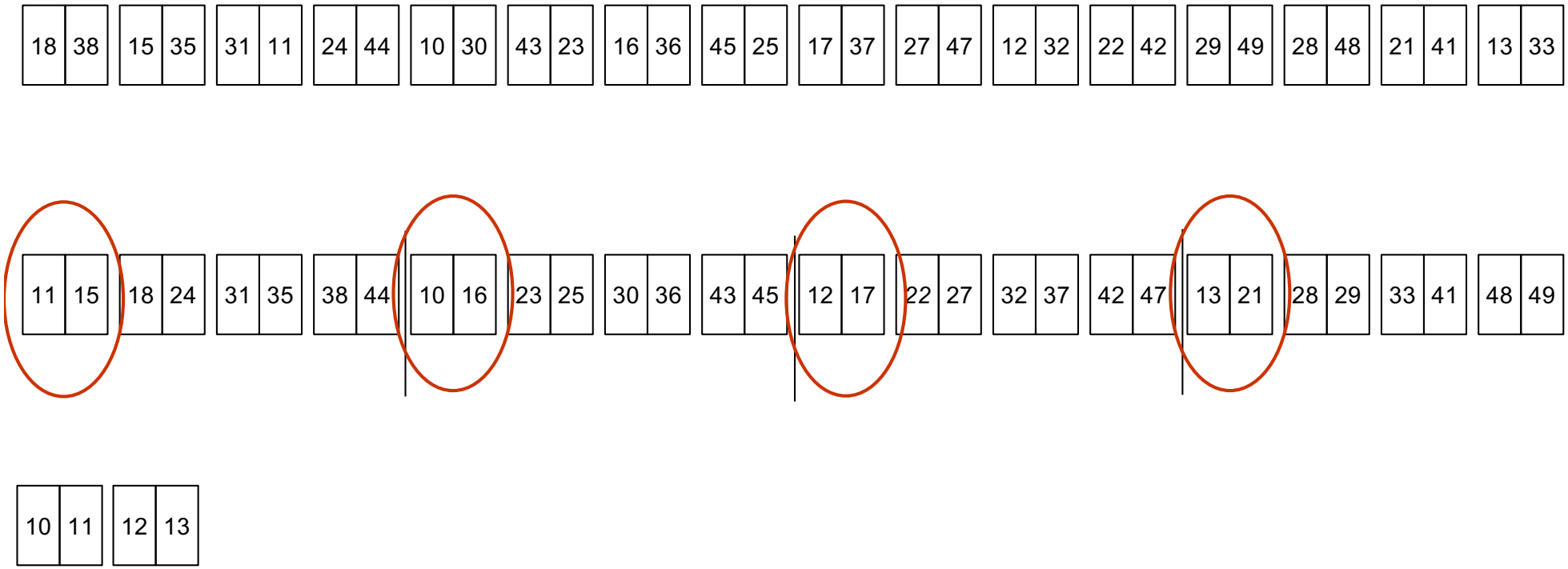


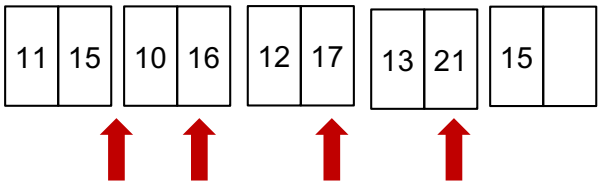
Secondo passo: merge a quattro vie



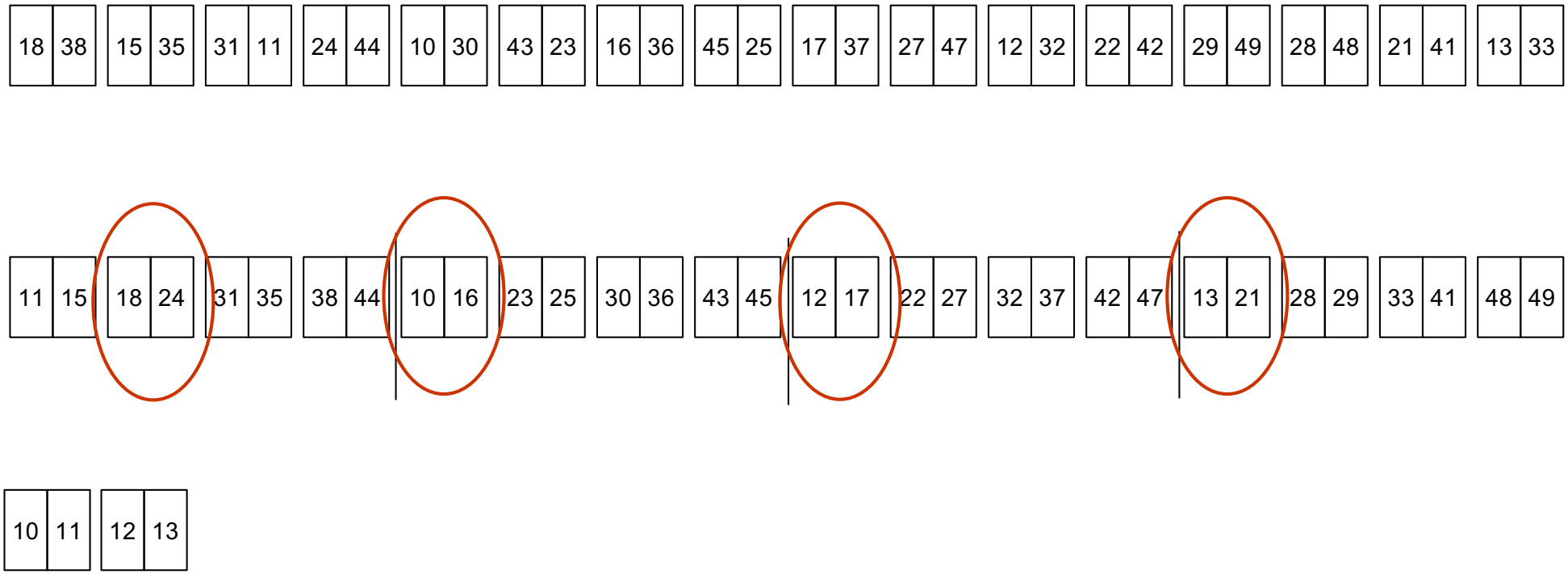


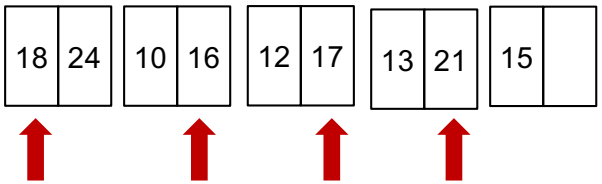
Secondo passo: merge a quattro vie



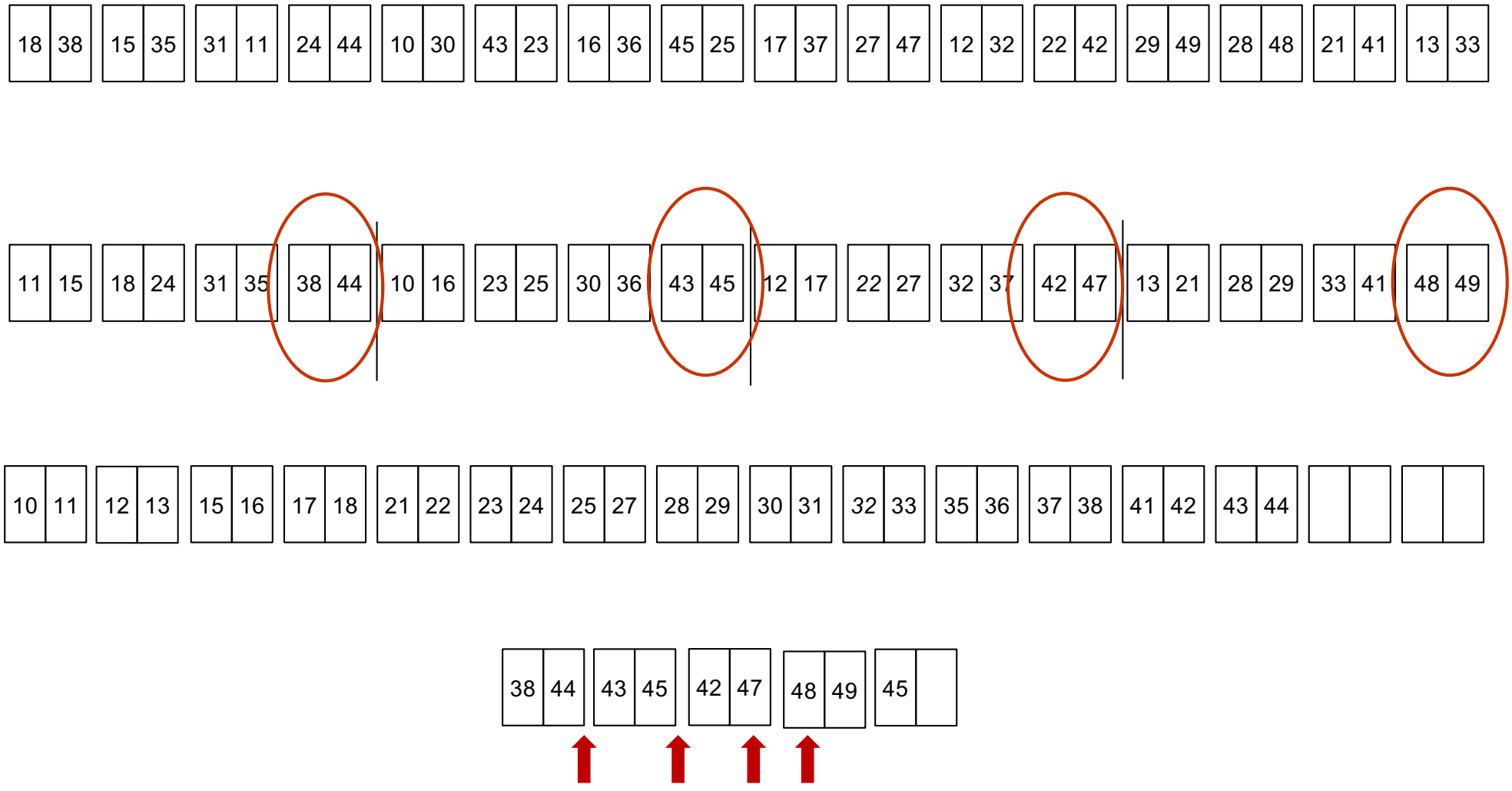


Secondo passo: merge a quattro vie

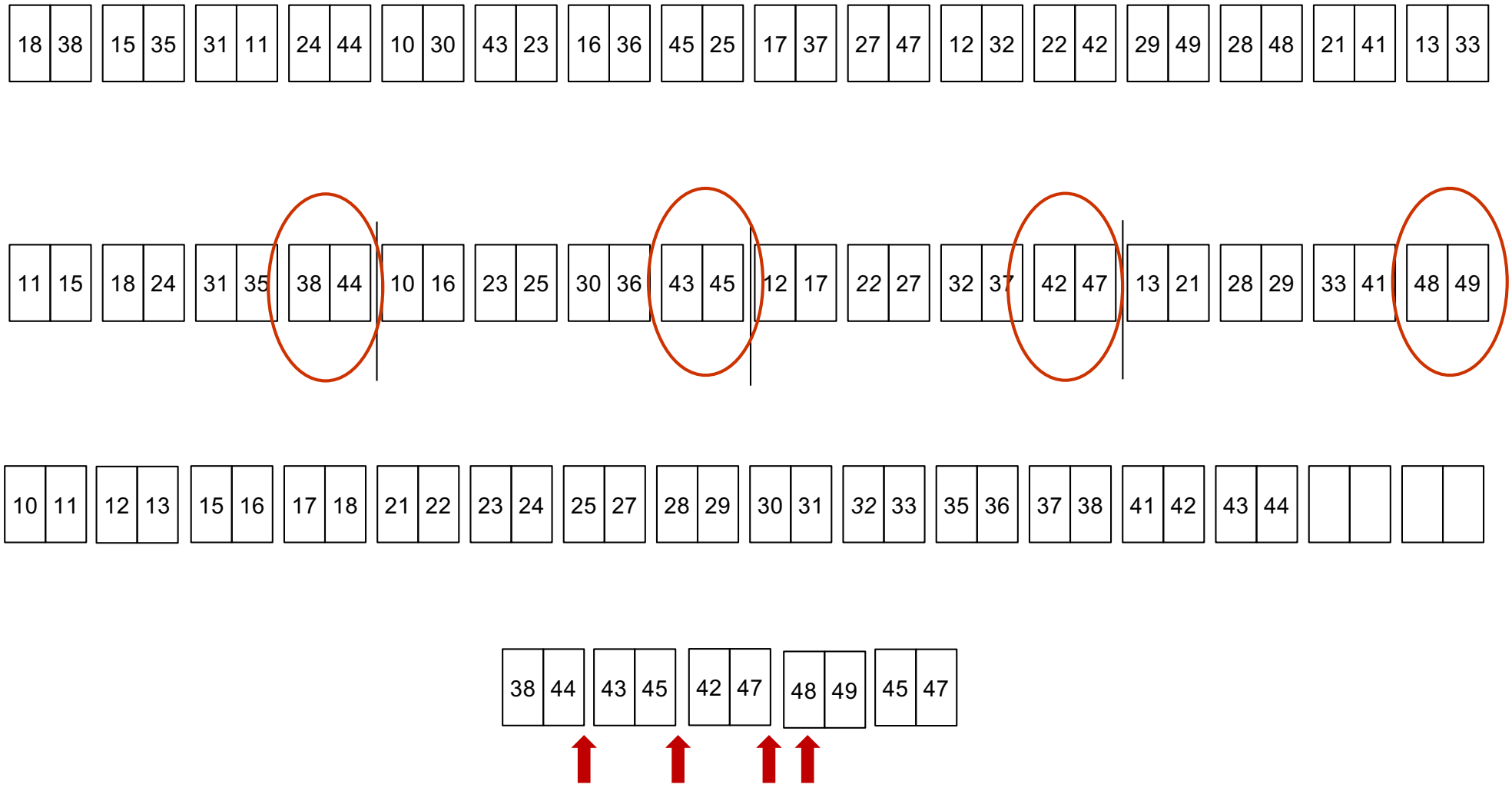




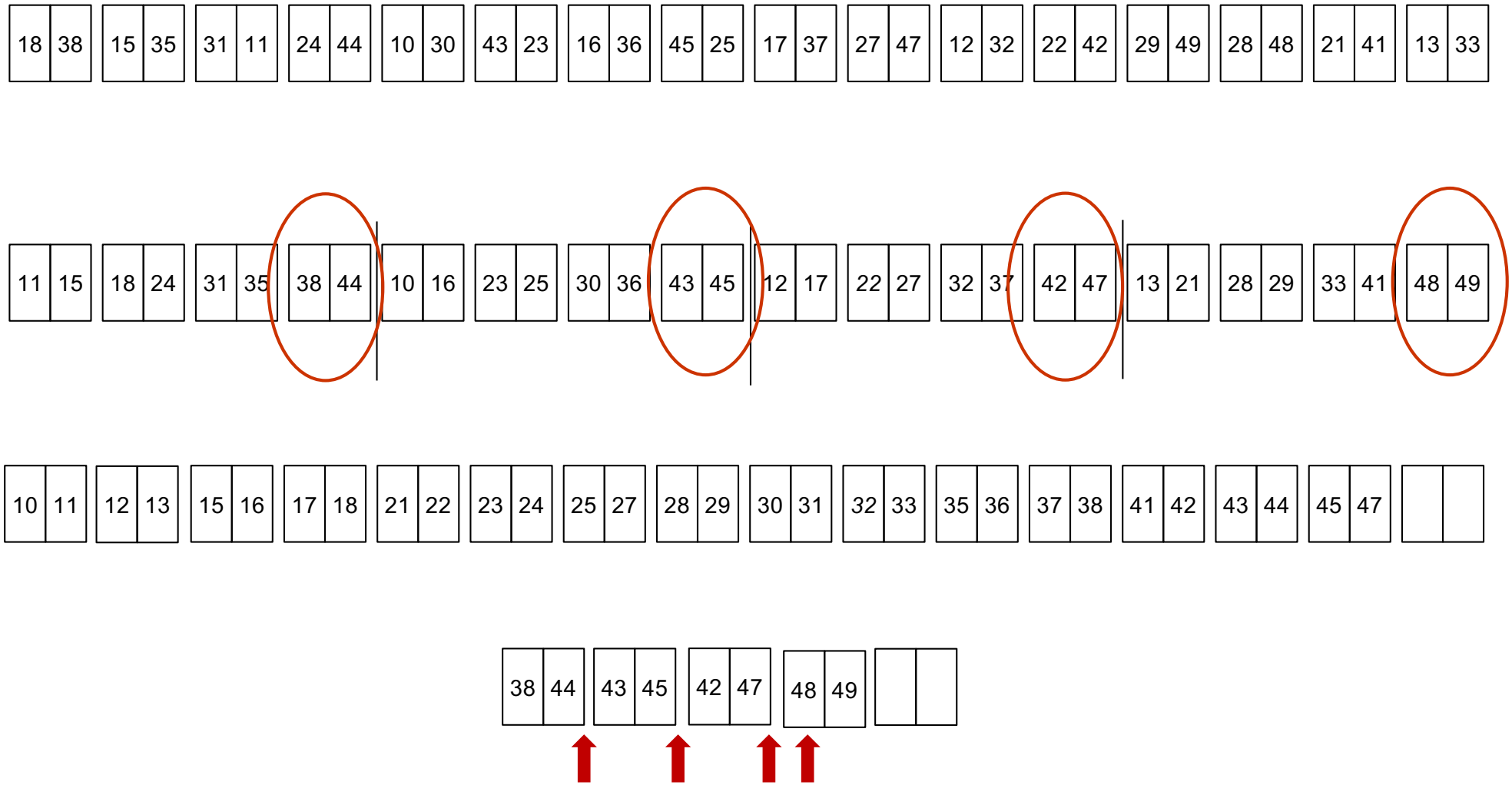
Secondo passo: merge a quattro vie



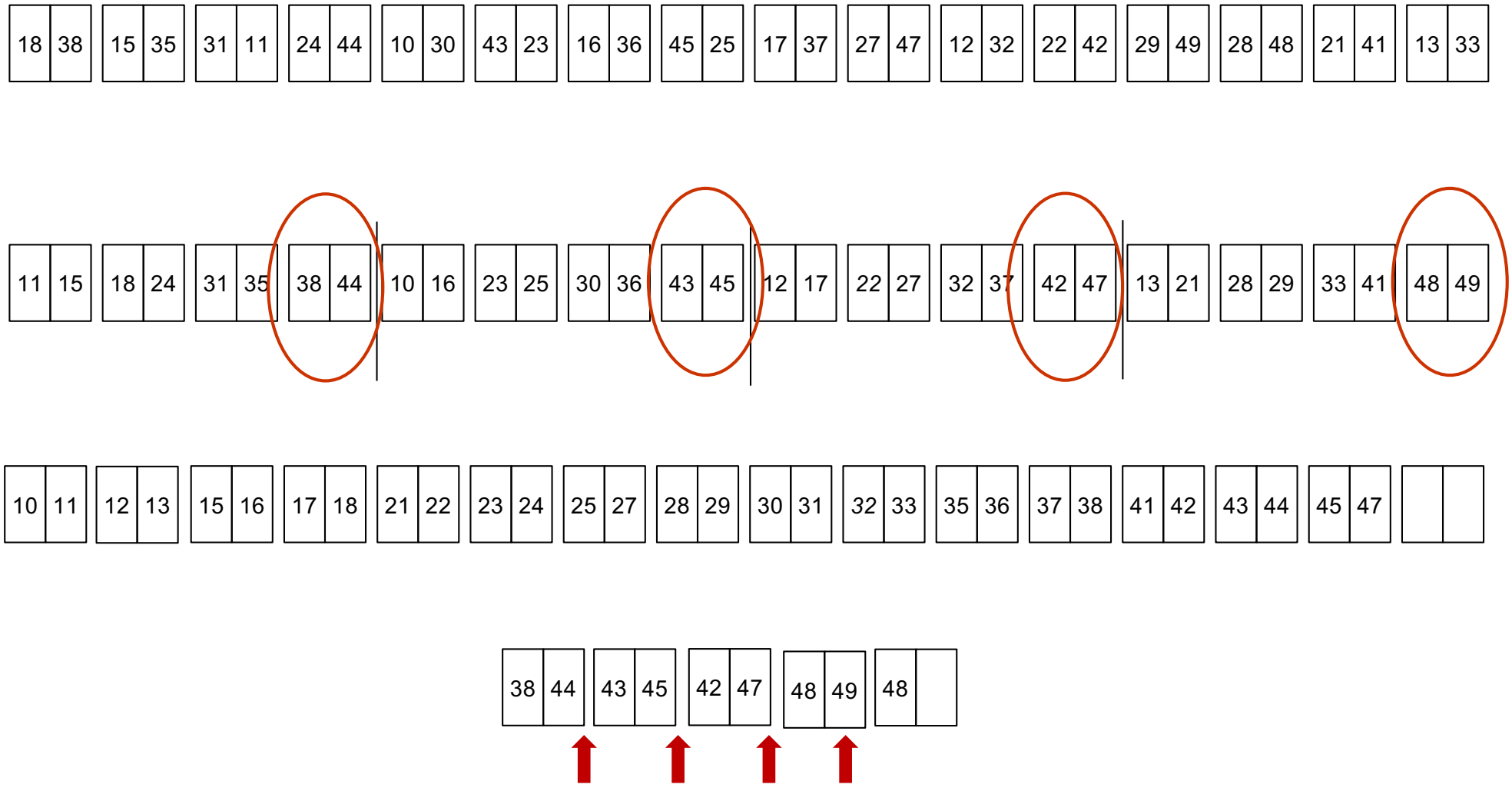
Secondo passo: merge a quattro vie



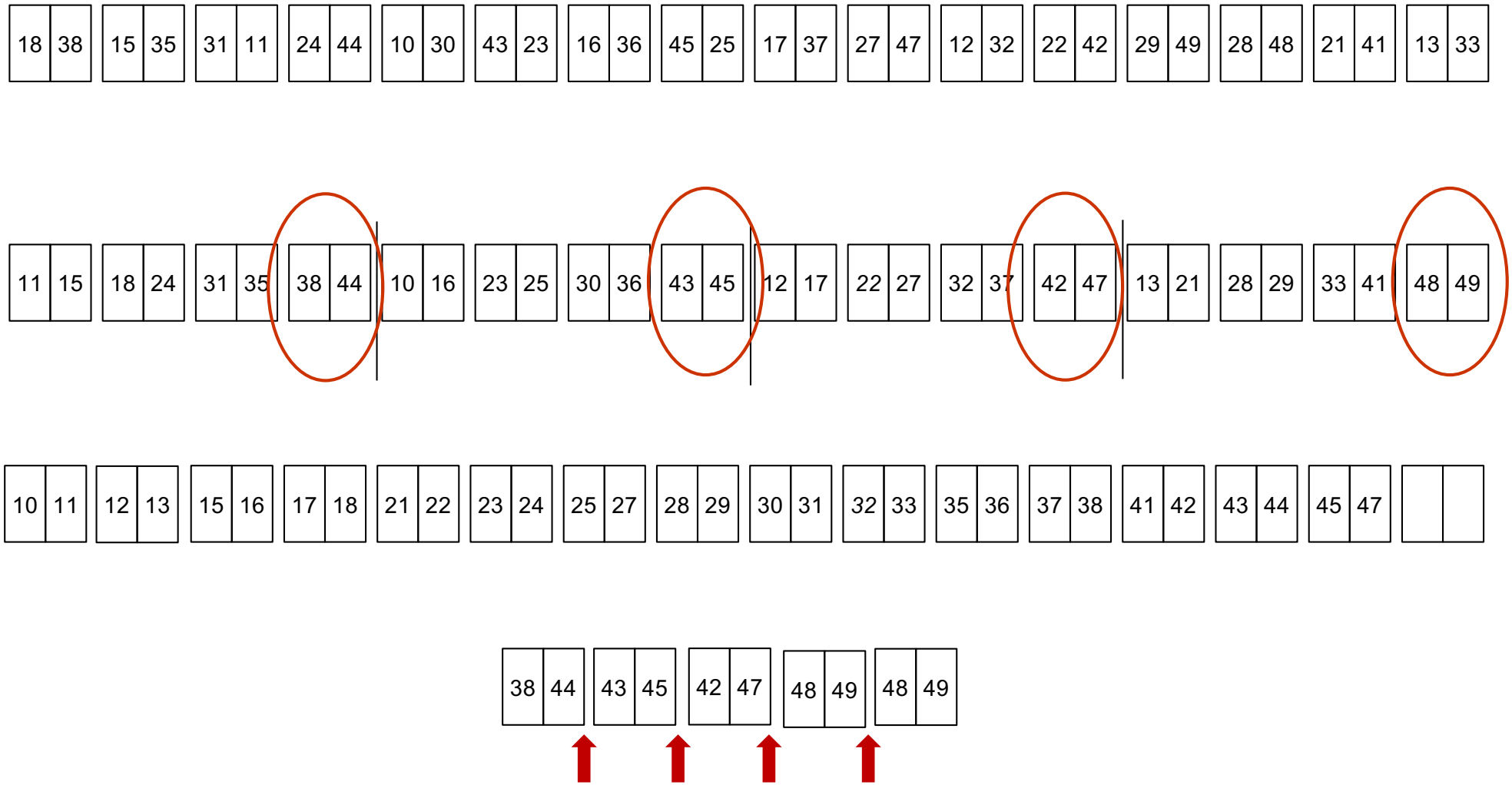
Secondo passo: merge a quattro vie



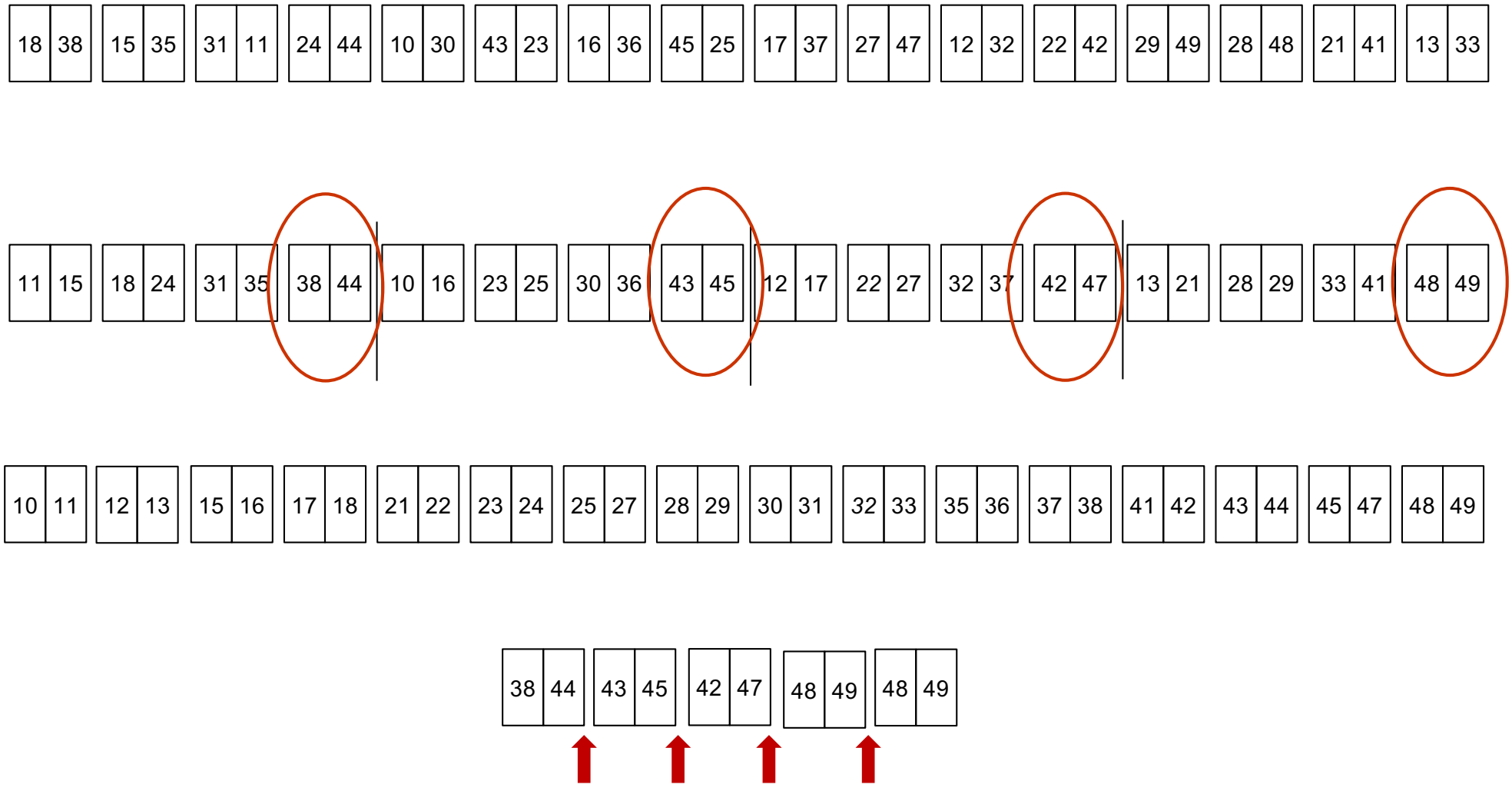
Secondo passo: merge a quattro vie



Secondo passo: merge a quattro vie



Secondo passo: merge a quattro vie



L'esempio "grande"

- Blocchi di 10KB
- 1.000.000.000 record di 100 byte: N = 10.000.000 blocchi
- Buffer disponibile di 100MB: 10.000 pagine

- Potremmo ordinare 10.000 blocchi alla volta e poi, avendo 1000 porzioni ordinate, fonderle con un merge a "1000 vie", quindi con un solo passo di merge!
- Per fare lo stesso basterebbero meno di 3200 pagine (radice quadrata di 10.000.000):
  - potremmo fare porzioni ordinate di 3200 pagine
  - merge a 3200 vie

Merge sort, quanti passi di merge?

- Con P pagine di buffer, possiamo
  - ordinare P blocchi
  - fondere (merge) P porzioni ordinate (run)
- Quindi, con P buffer possiamo ordinare con un solo passo di merge (preceduto da ordinamenti dei run di P blocchi) un file di P^2 blocchi:
  1. ordinamento di P run ognuno di P blocchi
  2. fusione di P run
- Sempre con P buffer possiamo ordinare con due passi di merge un file di P^3 blocchi
  1. ordinamento di P run ognuno di P blocchi
  2. fusione di P run (di P blocchi ciascuno) alla volta a formare run di P^2 blocchi
  3. fusione di P run (di P^2 blocchi ciascuno) alla volta a formare il file ordinato di P^3 blocchi

Merge sort, quanti passi di merge? (2)

- Con P buffer, possiamo
  - ordinare direttamente un file di P blocchi (una passata)
  - con un primo ordinamento e un merge (due passate), ordinare un file di P^2 blocchi
  - con un primo ordinamento e due merge (tre passate), ordinare un file di P^3 blocchi
  - …
  - con i passate, possiamo ordinare un file di P^i blocchi
- Dato un file di N blocchi, lo possiamo ordinare in i passate se
  - $P^i \geq N$ cioè se P è ≥ della radice i-esima di N

  Quindi il numero di passate necessario è il più piccolo i per cui risulta P ≥ della radice i-esima di N e il numero di buffer da utilizzare è proprio la radice i-esima di N (più precisamente, la sua parte intera superiore)

Merge-sort, in sintesi

- Se si riesce a fare in due passate, cioè un passo di merge (e non serve memorizzare il risultato) per un file di N blocchi
  - $3 \times N$
    - per procedere in due passate, serve un numero di pagine di buffer pari almeno alla radice quadrata di N

Esercizio

- Compito d'esame del 26/02/2013, esercizio 5

  https://tagd.inf.uniroma3.it/compitiPDF/20130226bdIItesto.pdf

  – 16 blocchi, 8 buffer
    - una passata (tutto in memoria) non basta (8 è minore di 16)
    - due (un ordinamento iniziale e un merge) bastano (8 è maggiore di 4) e si possono usare 4 buffer